Between Chance and Structure: Designing a Generative System for Ambient Music

University of Edinburgh

Chaoran Yu

MSc Sound Design

August

2026

Keywords: Constraint-based Generative Composition, Algorithmic Composition, Constrained Stochastic Processes, Markov Chains, Ambient Music, Lo-fi, Braindance, Scene-Based Sound Design, Max/MSP, VCV Rack

Acknowledgements

I would like to thank my supervisor, Dr Tom Mudd, for his guidance, feedback and support throughout this project. I am grateful to all the tutors who offered advice during its development, and to the technical staff and fellow students who provided practical help and responded to early versions of the system. Finally, I would like to thank my family and friends for their support throughout the research, development and documentation of the work.

Abstract

This project grew from two earlier attempts to build generative Techno and Drum & Bass patches. Both could produce effective short passages, but longer runs often lost a clear musical direction. I therefore shifted the work towards ambient and lo-fi material, retaining some of the uneven rhythmic behaviour of IDM and braindance.

The final system asks how far probability can vary local musical events before the larger form becomes difficult to follow.

The main patch runs in Max/MSP. I implemented synthesis in VCV Rack, where separate instruments produce the Pad, Melody and Drone. For each section, the harmony patch chooses a mode and saves two or three related chords,and then Melody, drums and environmental recordings are prepared by their own modules.
Continuity came from keeping those choices for long enough to hear them again. The five formal states decide when instrumental layers enter or leave, and the section memories repeat the current material. Two modes: Evolve runs until it is stopped. Timed builds an approximate form from the duration entered by the user.

Project Description

Item Listing

The submission contains:

*The final Max/MSP project for **Between Chance and Structure***

The associated VCV Rack synthesis patch

The complete sample and patch libraries required by the system

A formal demonstration video showing the system in operation

A technical walkthrough video about the internal Max/MSP and VCV Rack architecture

Stereo recordings of representative generated performances

System architecture diagrams and supporting screenshots

A setup and operating guide

A document listing the sources and usage information for external audio materials

The written project report

All software and supporting files are in the practical-work folder. The README covers software requirements, routing, file structure and startup. The formal video demonstrates the interface; the walkthrough follows the main Max/MSP and VCV Rack modules. The audio examples can be assessed without opening the patches.

I also recorded a 25-minute video continuous run to show that the system can sustain a complete performance. If you have interest:

https://drive.google.com/file/d/1TPtyGSVmoVBKu10N4AeWZmzZfzc1u64b/view?usp=drive_link (Accessed: 18 August 2026).

Recommended Viewing Order

View the formal demonstration first. Chapter 4 can then be read alongside the technical walkthrough.

Contents

Acknowledgements ii
Abstract iii
Project Description iv

1 Introduction 1
1.1 Background and Motivation 1
1.2 Prototype Development and Problem Definition 1
1.3 Research Question 2
1.4 Project Aims 2
1.5 Scope and Contribution 3
2 Context and Theoretical Framework 4
2.1 Generative and Algorithmic Composition 4
2.2 Chance, Probability and Constraint 4
2.3 Markov Models and Sequential Dependence 5
2.4 Memory, Repetition and Musical Identity 6
2.5 Ambient, Lo-fi and IDM Aesthetics 6
3 System Material Preparation and Methodology 7
3.1 Symbolic Representation and State Encoding 7
3.2 Authored Materials, Constraints and Iterative Design Method 7
3.3 Sample and Scene Design 8
3.4 VCV Rack Synthesis and Timbre Design 8
4 System Design and Implementation 9
4.1 Overall Architecture 9
4.2 Timing and Global Control 11
4.2.1 Global Clock Control 11
4.2.2 Global Mode: Evolve and Timed 12
4.3 Form and State-Machine System 13
4.3.1 Five-State Formal Model 14
4.3.2 Scene Selection and Orchestration Profiles 15
4.3.3 Voice Entry and Exit Planning 15
4.4 Harmony and Pitched Material 16
4.4.1 Harmonic System 16
4.4.2 Functional-State Transition 17
4.4.3 Chord-Root Selection and Diatonic Chord Construction 18
4.4.4 Controlled Section-Level Modulation 19
4.4.5 Section-Level Harmonic Memory 21
4.4.6 Pad Chord Grammar 22

4.4.7 Inversion and Minimum-Movement Voice Leading 24
4.4.8 Melody Generation, Phrase Memory, Density and Rest 25
4.5 Drum System, Probabilistic Percussion and IDM Layer 27
4.5.1 Pattern Generation and Section Identity 29
4.5.2 Drum State Coding and Conditional IDM Decision-Making 31
4.5.3 Sample Palette and Phrase Memory 32
4.6 VCV Rack Synthesis and Audio Routing 32
4.6.1 OSC Communication between Max/MSP and VCV Rack 32
4.6.2 Polyphonic Pad Synthesis 33
4.6.3 Melody and Drone Synthesis 34
4.7 Sample-Based Soundscape 36
4.7.1 Scene-Based Ambient Beds 36
4.7.2 Phase-Dependent Environmental Events 37
4.8 User Media, Mixing, Recording and Interface 38
5 Evaluation and Critical Reflection 38
5.1 Musical Results 38
5.2 Effectiveness of Structured Probability 39
5.3 Sound-Design Evaluation 39
5.4 Limitations 40
5.5 Future Development 40
6 Reflection and Conclusion 41
6.1 Answer to the Research Question 41
6.2 Creative and Technical Reflection 41
References 42
Appendix A: AI Use Statement 43
Appendix B: Third-Party Materials and Credits 44
Appendix C: Probability and Mapping Tables 45

1 Introduction

1.1 Background and Motivation

Generative music provided a practical model for this project. Fell and Gilmore (2012) examine music organised through compositional processes, and Collins (2008) discusses systems able of producing multiple musical outcomes with varying degrees of autonomy. I adopted this process-based view when designing the patch. Before playback, the user selects a scene and an operating mode. During playback, the Max/MSP patch generates harmonic, rhythmic and formal events from authored rules, weighted probabilities and stored musical material.

I chose Max/MSP because the timing, event logic, audio processing and interface could remain visible in one patch.

VCV Rack is used for synthesis. Max sends pitch, trigger and level messages by OSC; Rack turns them into sound and returns the audio for mixing.

1.2 Prototype Development and Problem Definition

The project started with two other patches, one techno and one drum and bass. I worked eight bars at a time. Usually that meant swapping a hi-hat or moving a fill until I liked the loop. Longer runs were different. No single choice was necessarily wrong. The choices simply knew nothing about one another.

To deal with that, the patch needed some way of listening to itself. It would have had to track spectral density and recognise when two parts were masking one another. Building that system would have taken more time than the project allowed. I kept both patches as studies and moved towards ambient music, where fewer elements change at once and a limited pool of material can carry more of the form.

A video of the earlier prototype: https://drive.google.com/drive/folders/1Jfri7rW8fHP9BasJHGoqyHKyGuTt0cQs?usp=drive_link (Accessed: 18 August 2026).

Ambient and lo-fi material allowed slower repetition, so they became the basis of the next version. IDM remained as a rhythmic influence. I listened closely to Boards of Canada and Aphex Twin, without trying to copy either artist. Sustained sounds could carry the form, and irregular percussion could add detail without demanding a new event at every moment. This also suited some of the roughness that remained in the rule-based patch.

1.3 Research Question

This project is guided by the following research questions:

How can structured probability be used to generate ambient music that remains varied while retaining coherent musical form?

I use structured probability to mean weighted choices made inside rules that I authored. These include Markov transitions, harmonic and rhythmic pools, phrase memory and the five formal states. The differences within the same run as well as between different runs will all lead to changes. I listened for coherence in three practical ways: the return of harmonic material, a readable order of instrumental entries, and audible changes in density or texture.

1.4 Project Aims

• Design a generative architecture across event, phrase, section and complete form.

• Use weighted choices in harmony, melody and rhythm. First-order Markov chains give each new event some relation to the one before it.

• To preserve continuity through stored musical material and rhythmic templates.

• To coordinate tempo, density, texture and layer entry with a scene-dependent state machine.

• To integrate sampling, Max/MSP processing and VCV Rack synthesis.

• To support Evolve and Timed operating modes.

• To provide scene and mode controls, imported-audio playback, monitoring and recording.

• To evaluate the system through repeated generation and critical listening.

1.5 Scope and Contribution

Three tasks are kept separate in the patch. Probability offers alternatives: memory holds a chord loop, phrase or beat.Formal control determines, when the texture expands and contracts.I kept revise the rules after hearing its effect.

And for pitched material, Max chooses modes and builds chords before sending Pad, Drone and Melody data to VCV Rack by OSC. Five custom sequencers (drum machine) play the kit, including a non-linear Closed Hat sequencers with irregular cycle lengths. The remaining soundscape comes from long beds system and occasional short Ambient Event system.

Evolve continues until the user stops it. Timed accepts an approximate duration, and an imported environmental recording can supply that duration instead. The imported file is included in the final mix. I intended these functions for installations and generative soundtrack work, and the recorded output also can also be taken into a DAW. This adds a touch of interesting practicality to this project.

2 Context and Theoretical Framework

2.1 Generative and Algorithmic Composition

Algorithmic composition can combine fixed rules with chance-based decisions. A generative work may follow the same process on each run and still produce a different musical result.
The terms overlap in this project.

Max/MSP carries out my written procedures. Their probabilistic branches change the finer musical events from one performance to another.

2.2 Chance, Probability and Constraint

In this project, chance means that a musical decision remains open until the system runs. Probability determines how likely each permitted result is. I use it at several levels, including chord roots, drum positions, melodic movement, sample choice, duration and section planning. Following Xenakis (1992), I treat these distributions as compositional material, although my implementation is practical and relatively small in scale.

The distributions are authored rather than neutral. Some choices are uniform, but many use repeated values or numerical thresholds to favour particular results. Repeating a value in a 'coll' increases its likelihood without fixing the outcome. This made the weighting easy to inspect and revise during listening tests. Weak or disruptive results could be reduced without removing variation from the system.

Probability alone does not define the musical result. Each draw takes place within a restricted set of possibilities. Carnovalini and Rodà describe rule-based music systems that use rules “as a final validation step, or to refine intermediate results” (2020, Section 3.2.3). This is close to the method used here. The patch narrows many choices before making a random draw. The active mode sets the available pitches, and the harmonic function selects a pool of chord roots. Drum hits use designated step positions. Scene settings define tempo, density and active instruments. Modulation is checked against harmonic function, timing and the available route. In some cases, the patch makes a choice first and discards it if it fails these checks. Carnovalini and

Rodà also describe constraints as a way of guiding the exploration of a defined conceptual space. In this project, that space is written by the composer and explored probabilistically by the program.

The report uses *structured probability* to describe this combination of weighted choice and authored restriction. It does not refer to one algorithm. It names a working method used across several modules. Some decisions are independent weighted draws, while others depend on what the system selected previously. Carnovalini and Rodà describe Markov chains as “well fit to describe melodies” because they model sequential events (2020, Section 3.2.1). In my system, this principle is applied to harmonic function, Pad chord shape and melodic movement. These sequential processes are examined in the following section.

2.3 Markov Models and Sequential Dependence

In the early version, each event was another draw from the same list. The result changed, but it did not seem to go anywhere. I replaced this with a first-order Markov chain. After an event is chosen, its state provides the weights for the next draw. Nothing before that state enters the calculation.

$$P(X_(n+1) = j | X_n = i, X_(n-1) = i_(n-1), \cdots, X_0 = i_0) = P(X_(n+1) = j | X_n = i)$$

If the chain is in state i, the outgoing weights from i determine the probability of reaching each possible state j.

A fixed list can preserve a general character, yet every draw begins from the same distribution. Giving each state its own successors creates short-term direction.
There are three such chains in the project. Harmony moves between Tonic, Predominant, Dominant and Color. The Pad chooses among six chord shapes. Melody uses five movement states: -2, -1, 0, +1 and +2 scale degrees. Carnovalini and Rodà (2020, Section 3.2.1) describe notes as states and derive transition counts from a corpus. I wrote the melodic movement weights by hand, then translated each move through the active mode.

I adjusted the weights by ear. For example, this led to a 0.60 probability of moving from Predominant to Dominant and a 0.75 probability of returning from Dominant to Tonic. The chain gives the harmony some local direction, although it remembers only the immediately preceding state.

2.4 Memory, Repetition and Musical Identity

One state is too little memory for a complete phrase. I therefore store the chord progression and melodic phrase for each section. Drum patterns stay in place until the next request, and the formal states operate over several sections. A new choice is delayed until the current motif or groove has had time to register.

2.5 Ambient, Lo-fi and IDM Aesthetics

Ambient music and experimental practice have often overlapped. For this project I joined ambient texture and lo-fi instability with rhythmic ideas from IDM and braindance, the term promoted by Rephlex Records for its strand of electronic music.Repetition and gradual change suited the generative method. Non-linear rhythm could disturb that surface without replacing it.
Boards of Canada informed my use of recurring motifs and local deformation; Aphex Twin was a reference for the drums.

3 System Material Preparation and Methodology

3.1 Symbolic Representation and State Encoding

I needed a simple way to store and pass musical material in Max/MSP, so most of it is represented numerically. Pitch is recorded as a scale degree, MIDI note or semitone offset, depending on the module. Rhythms are stored as step lists. Formal phases use integer IDs, and binary masks record which instruments are active.

Several lookups combine two values into a single 'coll' key. This only organises the data. Markov behaviour occurs elsewhere, when the saved current state changes the weighting of the following choice.

3.2 Authored Materials, Constraints and Iterative Design Method

I did not train the patch from a corpus. The transition weights, chord pools, rhythm templates, modulation routes and scene profiles were written by hand. Most of them went through several versions. After each listening pass, I located the smallest rule responsible for weak harmony, density or timing and changed that rule before running the patch again. The current numbers are a record of this process.

3.3 Sample and Scene Design

The environmental recordings were sorted into Forest, Rain and Steampunk. I first balanced their levels and basic tone in Ableton Live. Long recordings were selected for continuity and weak rhythmic definition. Short events had a different purpose, so I chose files with an immediately recognisable gesture.

For the IDM pool I prepared short Foley sounds, glitches, metallic hits, noise and reverse gestures with different attacks. Drum samples were chosen by instrumental role and by whether they remained audible inside dense patterns. I added EQ or reverb only where the sources needed balancing. A separate document lists their origins.

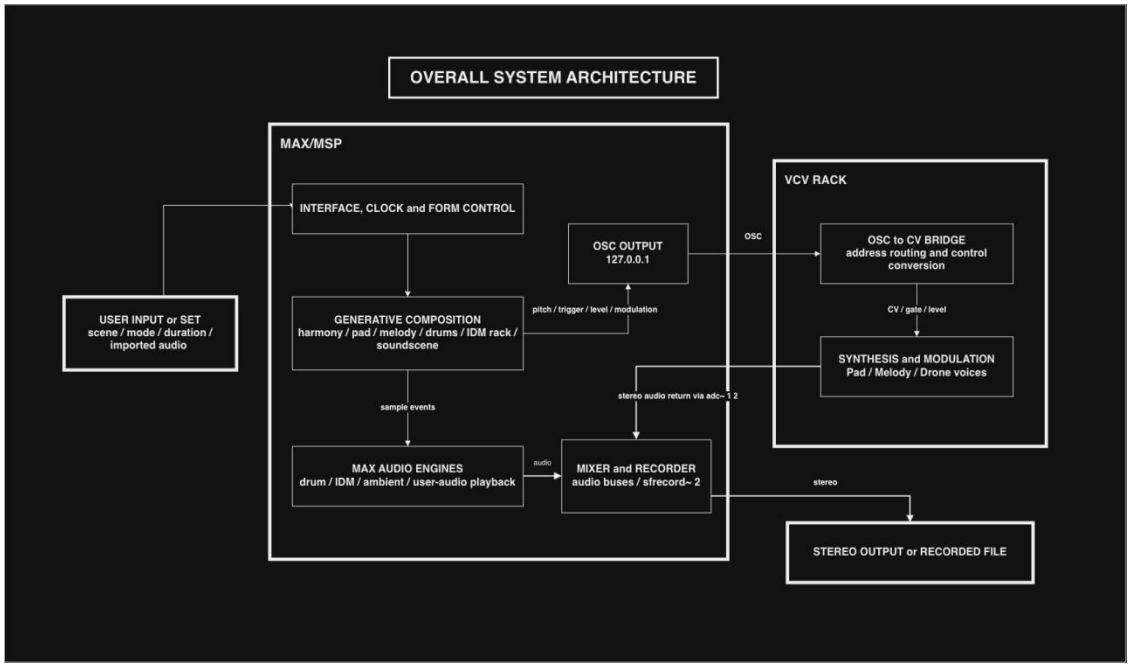
3.4 VCV Rack Synthesis and Timbre Design

Composition takes place in Max/MSP; the pitched voices are synthesised in VCV Rack. The active Rack file contains a four-voice Pad, one monophonic Melody voice and a Drone. Max sends pitch, trigger and level over OSC. The 'cvOSCcv' modules translate those messages into voltages and gates, and the stereo audio returns to Max for the final mix.

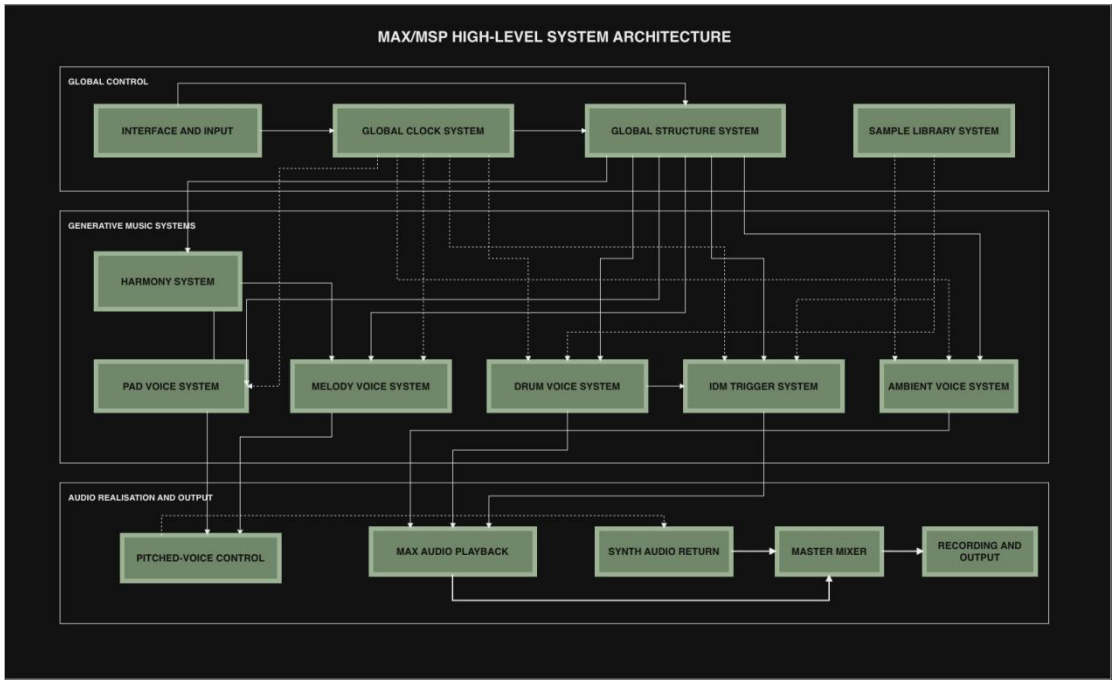
I could therefore alter an oscillator, envelope or effect without rebuilding the compositional patch. Section 4.6 follows each of the three signal paths.

4 System Design and Implementation

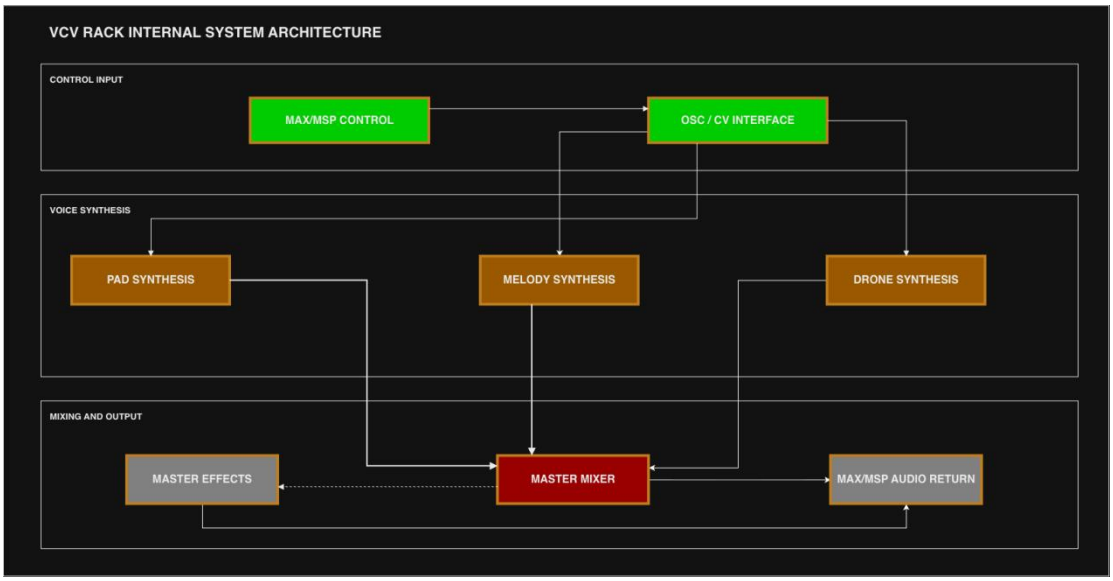
4.1 Overall Architecture



The overall system consists of two parts: Max/MSP and VCV Rack.



High-Level Architecture of the Max/MSP System



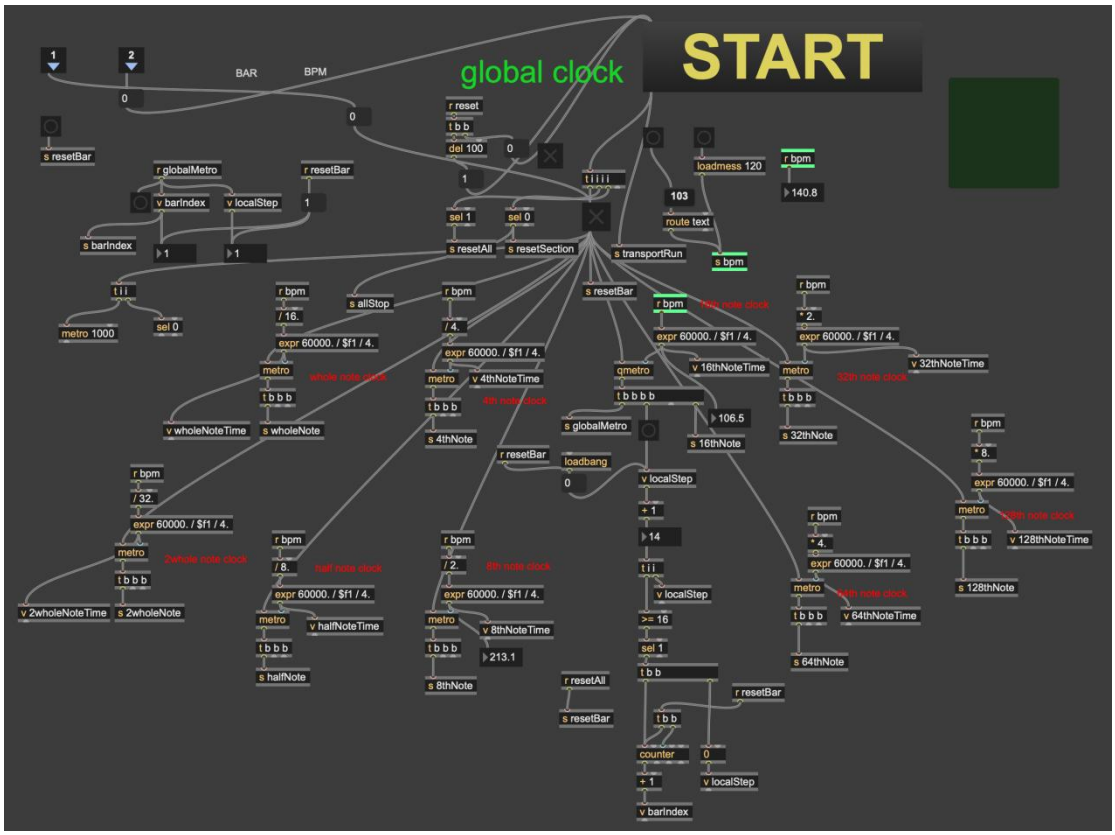
Architecture of the VCV Rack System

Max/MSP and VCV Rack divide the work between them. Max contains the interface, clock, form, generative rules, sample players, mixer and recorder. It sends pitch, trigger and level messages locally by OSC. In Rack, 'cvOSCcv' turns these messages into control voltages and gates. The Pad, Melody and Drone return through BlackHole and join the final Max mix.

Within Max/MSP, the patch is organised into control, generation and audio layers. Global controls coordinate the harmony, Pad, melody, drums, IDM events and environmental sound. The harmony engine supplies tonal material to the pitched voices, and drum-state data informs the IDM layer. Sample-based voices remain inside Max/MSP. Keeping rule generation and synthesis separate made both parts easier to revise. Section 4.6 describes the VCV Rack signal paths in detail.

4.2 Timing and Global Control

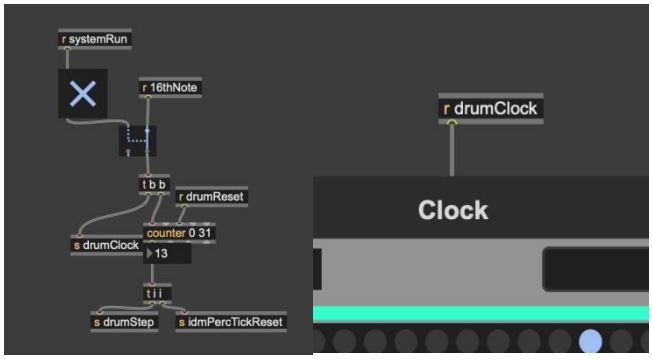
4.2.1 Global Clock Control



The patch uses one central BPM value. From this, it calculates the duration of a quarter note in milliseconds:

$$\textbf{Quarter-note duration (ms)} = 60{,}000 / \textbf{BPM}$$

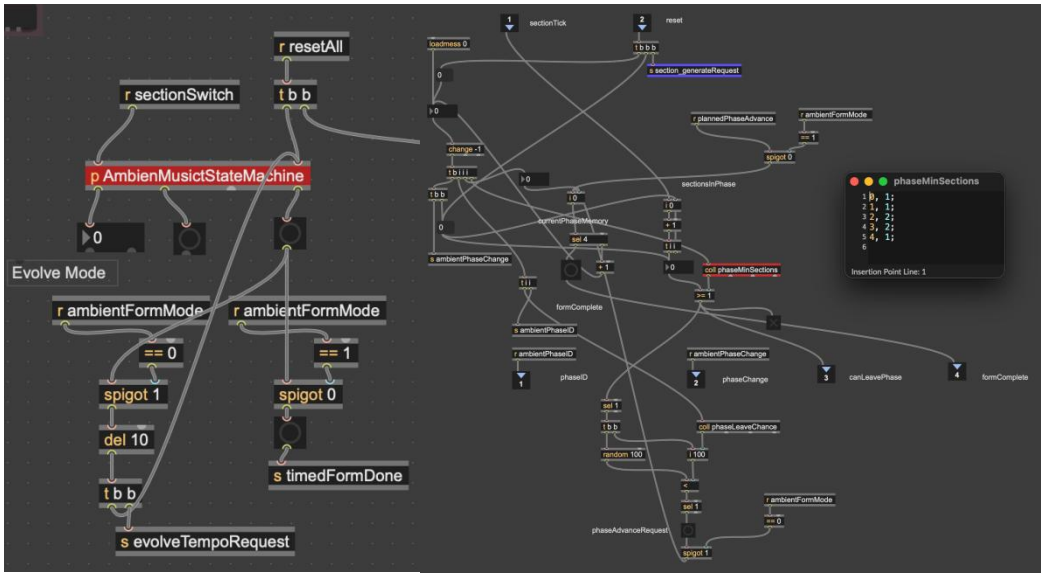
Named buses carry pulses from two whole notes down to 128th notes. Examples include 'wholeNote', '4thNote', '16thNote' and '64thNote'. I used the slow buses for form and harmony. Drum sequencing, especially the hi-hat parts, uses the finer divisions.



'drumStep' advances the drum grid. 'barIndex' keeps the position in the larger cycle, and 'drumSectionStart' requests a new drum pattern at a section boundary. Harmony and melody have separate request and readiness messages, but all of these decisions refer to the same metrical clock.

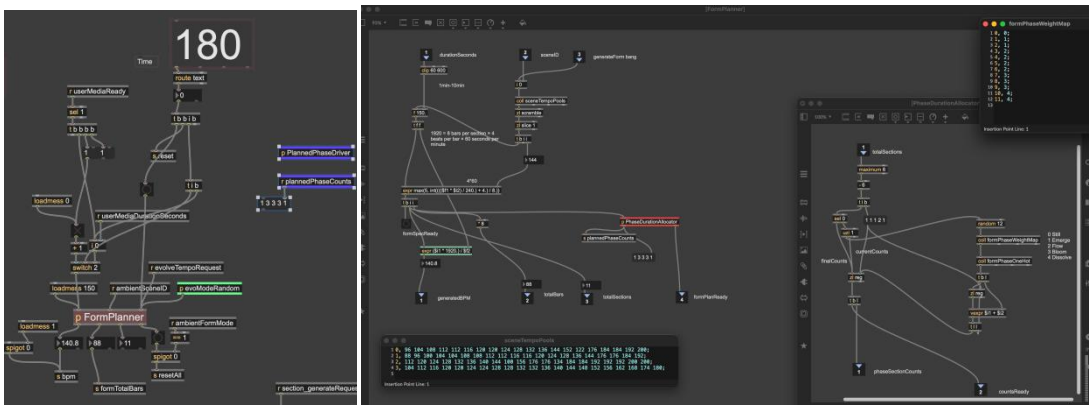
4.2.2 Global Mode: Evolve and Timed

The patch has two operating modes.



Evolve mode

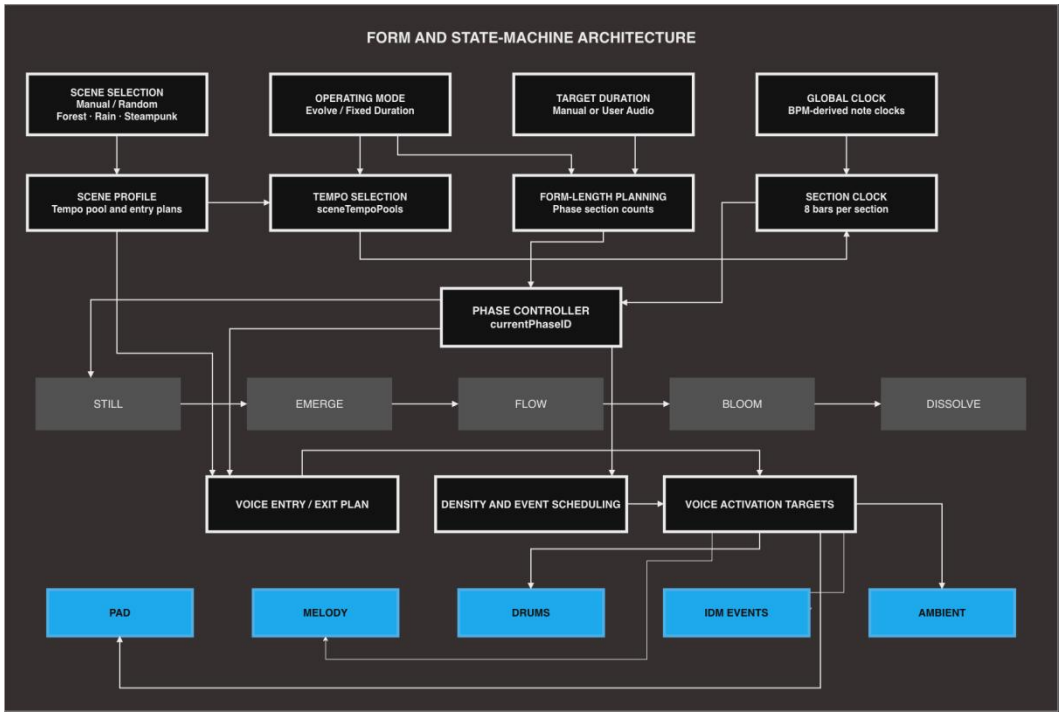
Evolve mode has no fixed endpoint. It moves through the five formal states and begins another cycle after DISSOLVE, with a new scene-dependent tempo requested for the next cycle. Minimum section lengths prevent the form from changing too quickly.



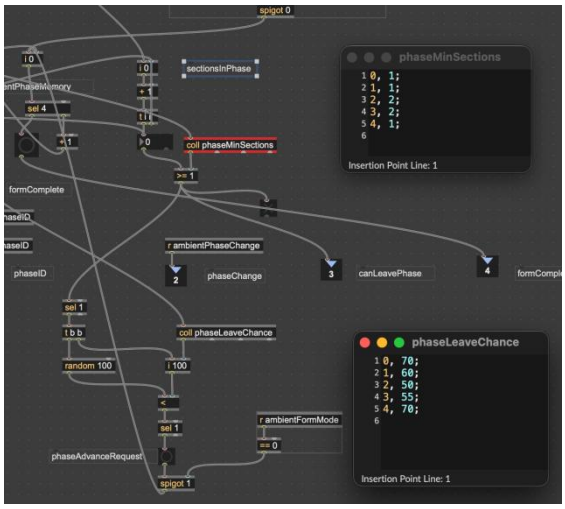
Timed mode

Timed mode begins from a duration entered by the user or read from an imported audio file. After the tempo has been selected, the system estimates the available number of sections and distributes them across the five formal states. The final DISSOLVE section is placed near the requested endpoint. Both modes use the same state, orchestration and voice-control modules; only the method of scheduling the form changes.

4.3 Form and State-Machine System



4.3.1 Five-State Formal Model



The formal model uses five ordered states: STILL (0), EMERGE (1), FLOW (2), BLOOM (3) and DISSOLVE (4). The current ID is distributed through 'ambientPhaseID'. STILL keeps only a small number of layers active. EMERGE introduces additional voices, followed by the main arrangement in FLOW. BLOOM has the highest instrumental and event density. DISSOLVE removes layers from the mix.

One section lasts eight bars. State changes are evaluated only at section boundaries. In Evolve mode, each state has a minimum duration. Once that duration has passed, a leave probability is tested at every subsequent boundary. A successful test advances the form by one state. After DISSOLVE, the form returns to STILL and requests a new tempo without changing the selected scene.

Timed mode allocates the five states before playback. 'FormPlanner' accepts a duration between 60 and 600 seconds and estimates the number of eight-bar sections:

Number of sections = max(5, floor(((duration × BPM / 240) + 4) / 8))

Each state receives at least one section. Additional sections are distributed through authored weights, with FLOW and BLOOM usually receiving more time than STILL. The complete allocation table is included in Appendix C. 'PlannedPhaseDriver' advances the states according to this plan. After the final DISSOLVE section, the voice mask is cleared and Timed mode stops the transport.

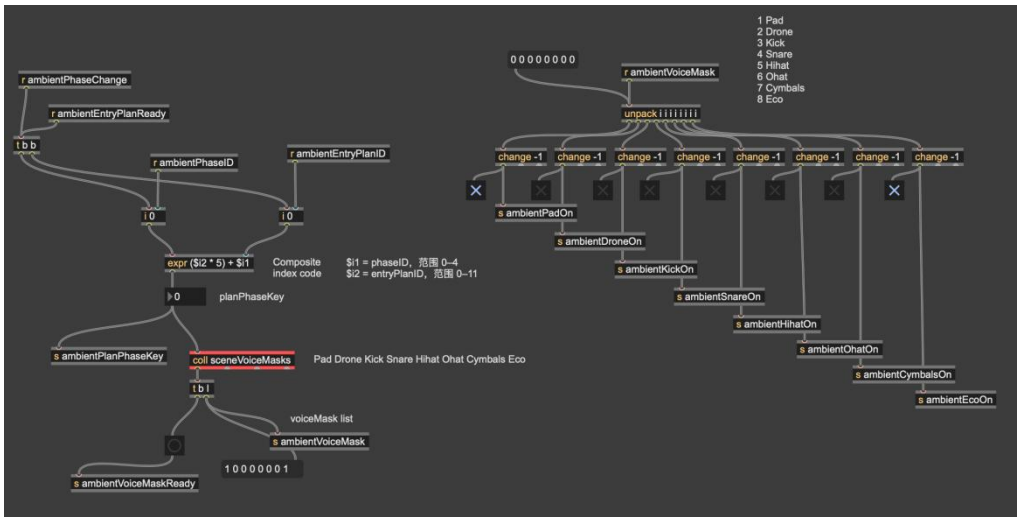
4.3.2 Scene Selection and Orchestration Profiles

The system contains three scene profiles: Forest (0), Rain (1) and Steampunk (2). A scene may be selected manually. At the beginning of an Evolve performance, the system can also choose one at random. Timed mode retains the user's selection.

A scene limits which orchestration plans can be selected. It does not fix a complete arrangement. 'SceneEntryPlanner' chooses among three plans assigned to that scene, and each plan stores an eight-value mask for every formal state. The mask addresses Pad, Drone, Kick, Snare, closed hi-hat, open hi-hat, cymbals and Eco, the IDM percussion layer.

Forest and Rain usually open with fewer layers. Steampunk can introduce percussion earlier. Scene ID also chooses the tempo pool and ambient-bed indices, and it biases Eco density. Appendix C gives the plan allocation and all voice masks.

4.3.3 Voice Entry and Exit Planning



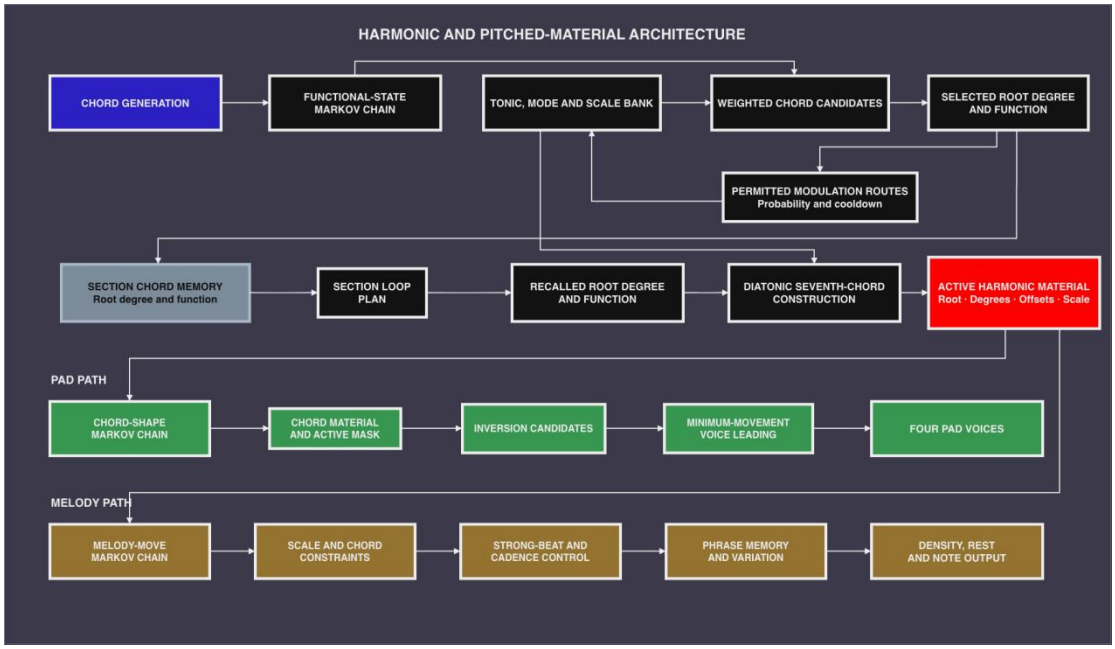
Every phase change combines the current entry-plan ID with the phase ID:

Key = entryPlanID × 5 + phaseID

That key reads one eight-value mask from 'sceneVoiceMasks'. After 'unpack', each value reaches its corresponding voice control. Selecting another entry plan changes the order of appearance; the five formal phases themselves stay in the same order.

DISSOLVE uses a separate exit procedure. 'RandomVoiceExitPlan' chooses one of 32 exit-count patterns and shuffles the eight voice IDs. On successive bars, 'VoiceExitMaskAccumulator' clears the chosen values and keeps them off for the rest of the section. Evolve starts this procedure on entry to DISSOLVE; Timed starts it during the last planned section. Melody is controlled elsewhere and does not belong to this mask.

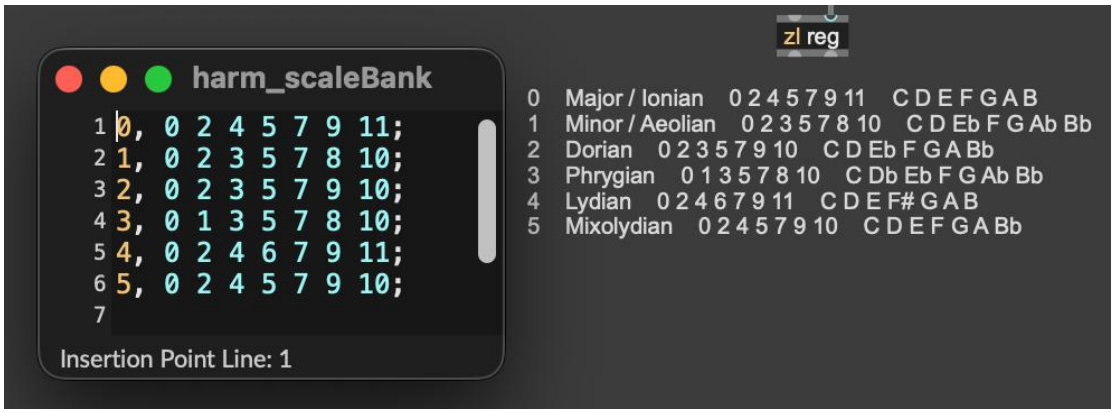
4.4 Harmony and Pitched Material



In the harmony patch, I generate the material first and store it for playback. 'p Harmony' uses the current tonic and mode to produce a chord-root degree and its harmonic function. After two or three results are written to 'sectionChordMemory', the eight-bar plan keeps calling those entries. They remain there until another generation request is made. The Pad and Melody parts follow the harmony being recalled at that moment.

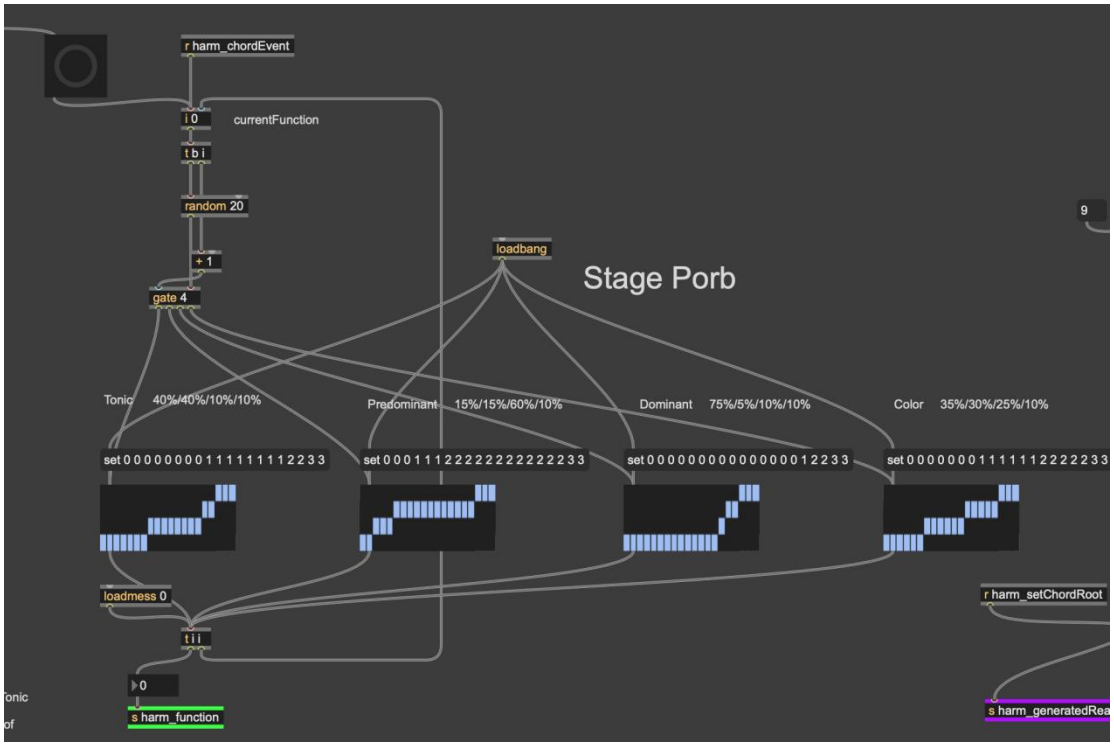
4.4.1 Harmonic System

Three values describe the tonal context: 'harm_tonicPC', 'harm_modeID' and 'harm_scale'. Tonic pitch class uses 0 to 11. Mode ID addresses one of the six seven-note entries in 'harm_scaleBank'.



'harm_scale' translates scale degrees into semitone intervals. For register, 'harm_tonicRef' adds 60 to the tonic pitch class. The saved patch opens on tonic pitch class 2 and mode ID 2, so its initial context is D Dorian. A later modulation may replace both values.

4.4.2 Functional-State Transition



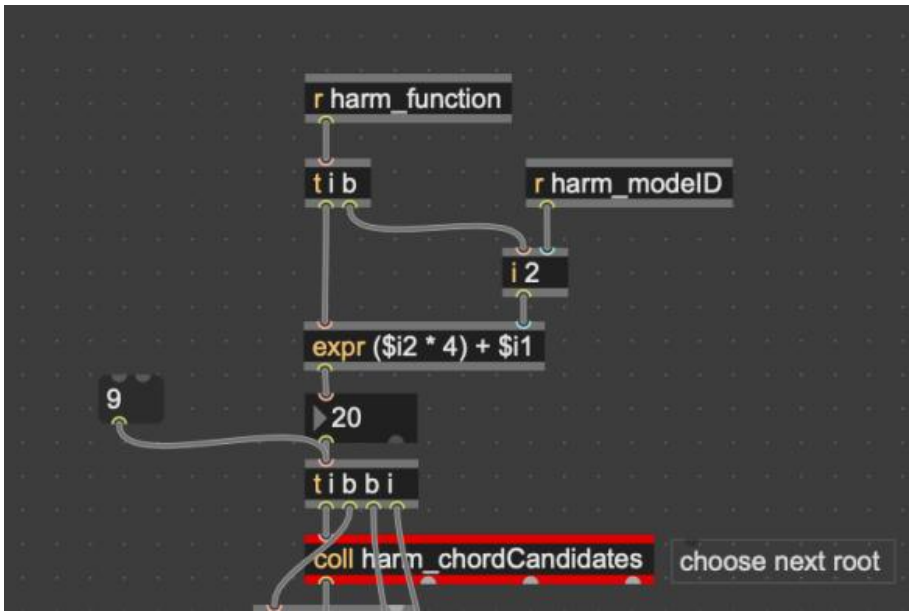
A chord event begins by choosing a function: Tonic (0), Predominant (1), Dominant (2) or Color (3). The present function opens one outlet of 'gate 4', selecting a 20-item successor list loaded by a 'set' message. 'random 20' reads one position. Repeated IDs therefore weight the result in steps of five percentage points. The chosen ID becomes the new function and is sent through 'harm_function'.

Together, the four successor lists make a hand-authored first-order Markov chain. Tonic generally stays stable or moves towards Predominant. Predominant leans towards Dominant, which most often returns to Tonic. Color has a looser set of successors. Appendix C gives the exact transition values.

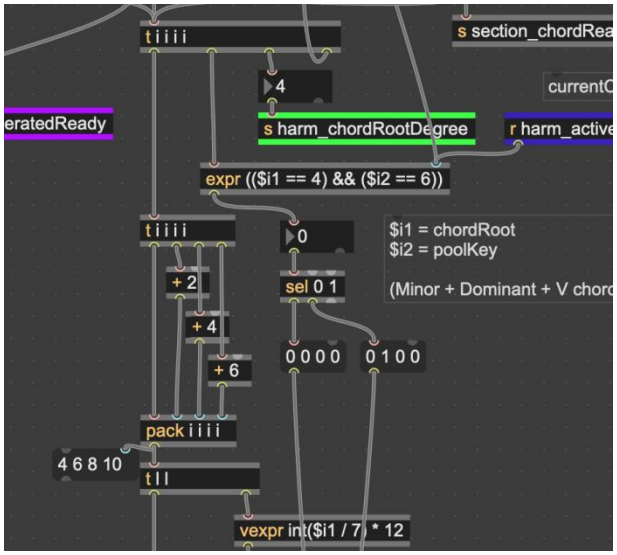
The function ID is then combined with the current mode ID:

candidatePoolKey = modeID × 4 + functionID

The combined key addresses 'coll harm_chordCandidates' and returns the root degrees allowed for that mode and function.



4.4.3 Chord-Root Selection and Diatonic Chord Construction



'coll harm_chordCandidates' contains 24 root-degree pools, with four harmonic functions for each of the six modes. I weighted a degree by writing it more than once. For example, the Major-Tonic list is 0 0 0 5 5 2: degree 0 occupies three of the six entries, degree 5 occupies two, and degree 2 occupies one. Appendix C lists all 24 pools.

A second test deals with immediate repetition. On 75% of chord events, 'zl filter' removes the root that is already sounding. The other 25% leaves it available. 'zl scramble' changes the candidate order, after which 'zl slice 1' takes the first degree.

The selected root is expanded as:

root, root + 2, root + 4, root + 6

From the chosen root, the patch adds scale-degree offsets 0, 2, 4 and 6. These are stacked diatonic thirds. Each degree is read through 'harm_scale', and every complete octave adds twelve semitones. 'harm_chordOffsets' carries the resulting interval list.

One rule changes the Minor-Dominant case. When the root is degree 4 and the pool key equals 6, the third is raised by one semitone before Pad voicing begins. The result is a dominant seventh chord.

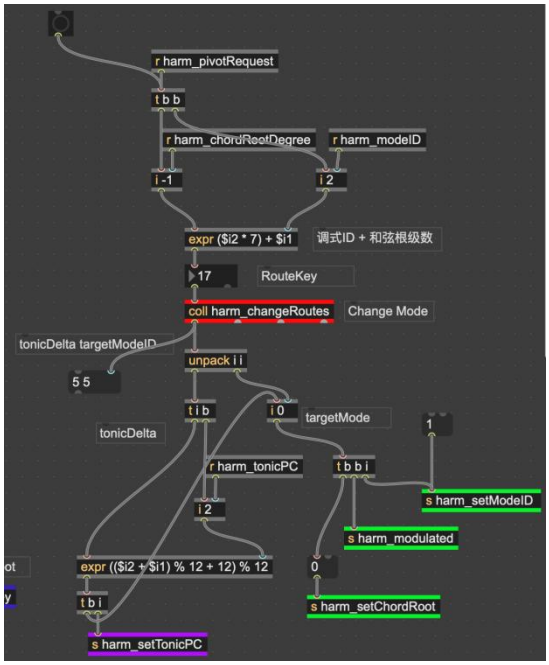
For example, D minor produces A-C-E-G at first. Raising C to C♯ gives A-C♯-E-G and restores the missing leading tone.

4.4.4 Controlled Section-Level Modulation

'harm_pivotRequest' comes from two parts of the patch. During section generation, 'p changeHarm' sends a request when 'random 100' passes '< 30', giving this route a 30% chance. The second route is inside 'p Harmony'. Its 15% test is reached only after eight to twelve chord events, with 'harm_modeLock' at 0 and the current function set to Tonic or Color.

For either route, mode ID and chord-root degree form the lookup key:

routeKey = modeID × 7 + chordRootDegree



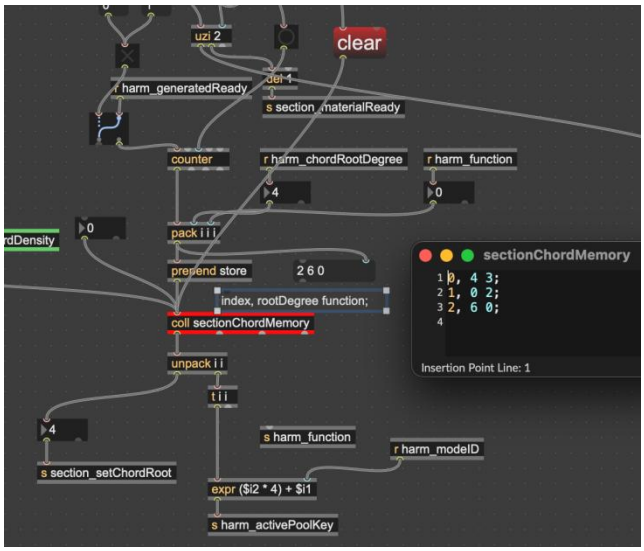
The 30 rows in 'coll harm_changeRoutes' are divided evenly across the six modes. When the calculated key exists, the row returns a tonic interval and target mode:

newTonicPC = (currentTonicPC + tonicDelta) mod 12

When a lookup succeeds, the target mode is loaded and the chord-root degree returns to 0, so the pivot chord becomes the new tonic. If no matching entry is found, the tonal context stays as it is. The complete route list is included in Appendix C.

4.4.5 Section-Level Harmonic Memory

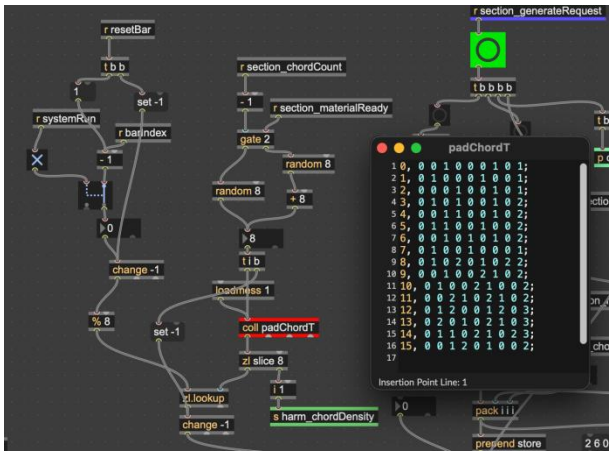
'section_generateRequest' first clears 'coll sectionChordMemory' and resets its write index. The pair 'random 2' and '+ 2' chooses whether the section will keep two or three chords. 'uzi' then requests exactly that many events from 'p Harmony'.



Each 'harm_generatedReady' event stores three values:

memory index, chord-root degree, harmonic-function ID

The memory holds symbols rather than completed MIDI notes. Playback rebuilds the pitches from the mode and scale that are active at that time.

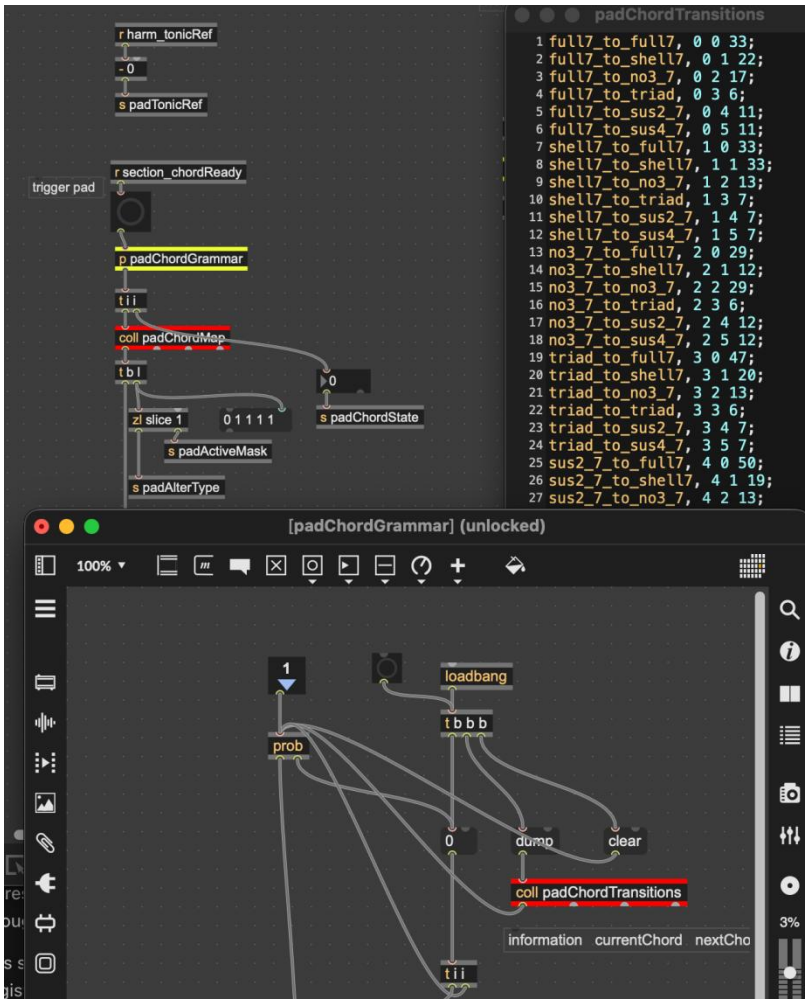


Once generation is complete, 'section_materialReady' chooses a row from the 16 templates in 'coll padChordT'. Rows 0 to 7 expect two saved chords;

rows 8 to 15 expect three. Eight values address the chord memory once per bar, followed by a ninth value for 'harm_chordDensity'.

For playback, 'barIndex' is reduced by one and wrapped through '% 8'. This gives a zero-based position in the selected template. The corresponding memory row restores function and root degree through 'section_setChordRoot'. When neighbouring bars call the same chord, 'change -1' blocks the duplicate trigger. The progression remains in place until a new section request.

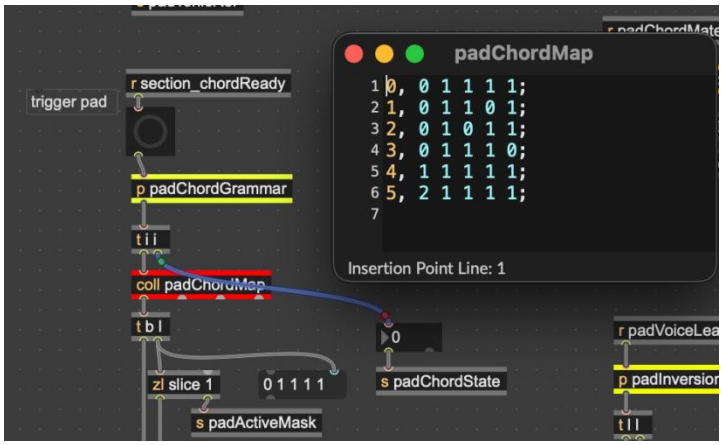
4.4.6 Pad Chord Grammar

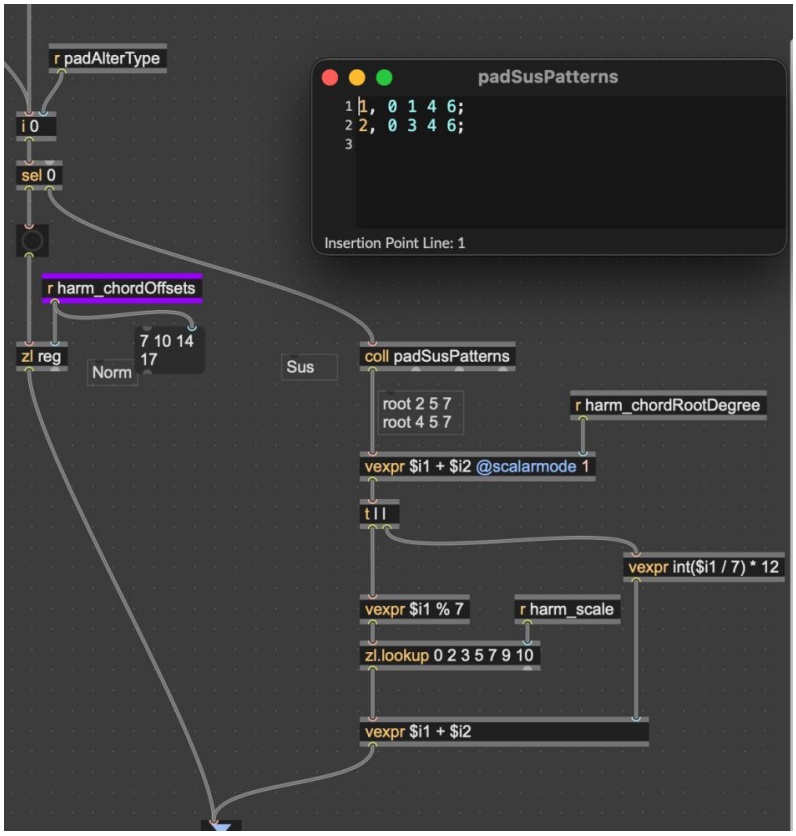


The Pad receives a root and diatonic seventh chord from the harmony engine. Every 'section_chordReady' message also advances 'p padChordGrammar', a first-order chain written in 'prob'. Its six states are 'Full7', 'Shell7', 'No3_7',

'Triad', 'Sus2_7' and 'Sus4_7'. I weighted the chain towards complete seventh chords and used the reduced or suspended states as shorter departures. Appendix C, Table C.1 gives the matrix.

'prob' keeps the selected state internally. Its right outlet is used only as a safeguard: an undefined state resets the chain to 0, 'Full7'.

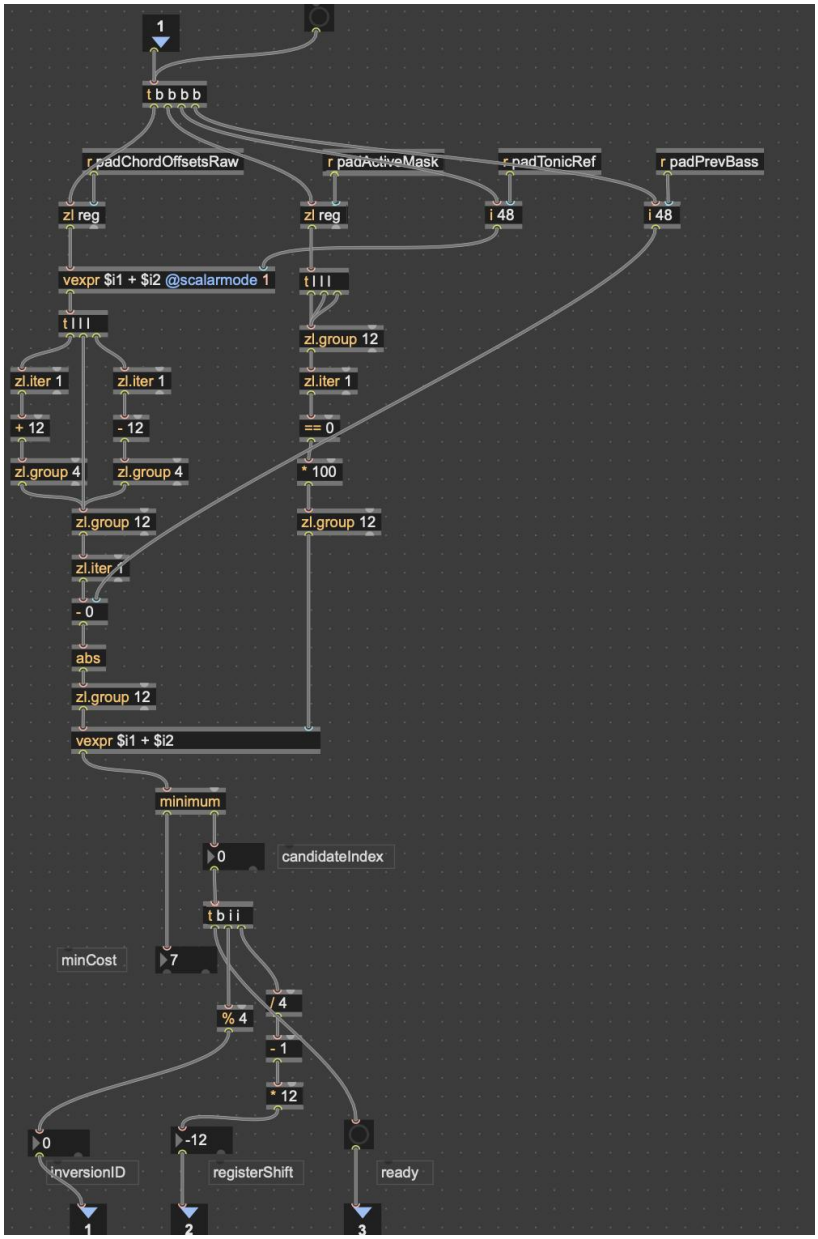




'coll padChordMap' turns the state into an alteration type and a four-value Boolean mask ordered root, third, fifth, seventh. Zero removes a tone. States 0 to 3 keep the intervals from 'harm_chordOffsets'. For states 4 and 5, 'coll padSusPatterns' substitutes [0, 1, 4, 6] or [0, 3, 4, 6]. Reading those degrees

through 'harm_scale' keeps either suspension inside the current mode. Appendix C, Table C.2 lists the state mapping.

4.4.7 Inversion and Minimum-Movement Voice Leading



After the chord material and activity mask are prepared, 'p padVoiceLeadSelector' tests twelve possible bass-note positions. It adds 'padTonicRef' to the four chord offsets and places each tone at three registers: −12, 0 and +12 semitones.

Each candidate is compared with 'padPrevBass'. Its cost is the absolute distance from the previous bass note, with an additional penalty of 100 when the corresponding chord tone is inactive. The 'minimum' object returns the candidate with the lowest cost. Its index determines both the inversion and register:

inversion ID = candidate index modulo 4

register shift = (integer division of candidate index by 4 – 1) × 12

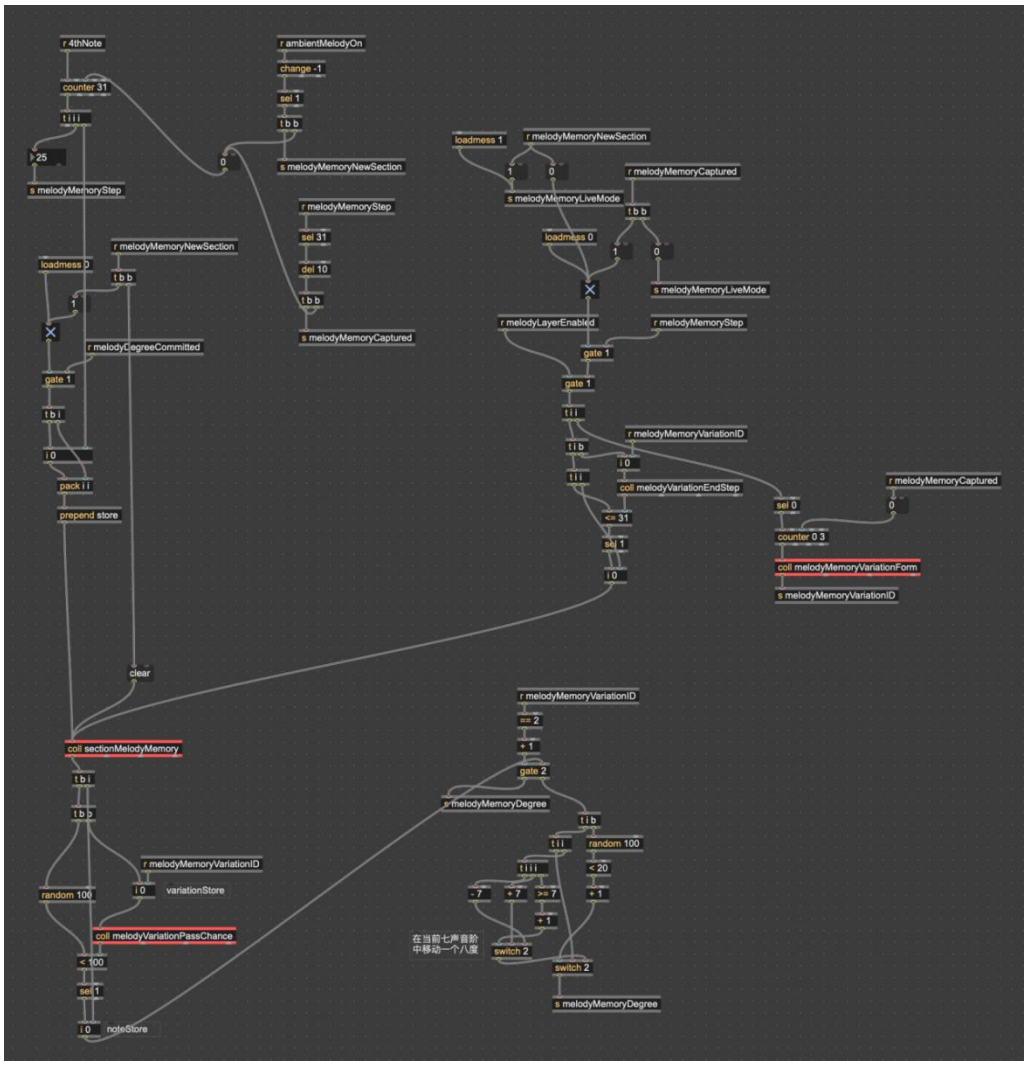
'p padInversionBuilder' reads the selected ordering from 'coll padInversionMap'. Two 'zl.lookup' objects apply the same ordering to 'padChordOffsetsRaw' and 'padActiveMask'. Octave offsets, 'padTonicRef' and 'padRegisterShift' are then added to form the final MIDI voicing. The reordered mask is retained as 'padVoicingMask'.

The first note of the completed voicing becomes the next 'padPrevBass'. The procedure minimises movement of the lowest active chord tone. It does not calculate the combined movement of all four voices.

4.4.8 Melody Generation, Phrase Memory, Density and Rest

'p MelodyPresenceDirector' keeps Melody off in STILL. It tests entry in FLOW (22%) and BLOOM (44%). When a phrase is requested, a sixteen-step quarter-note grid chooses one of 24 rhythm patterns. I divided the patterns into sparse, medium and dense groups, and 'coll melodyDensityByPhase' changes their weighting with formal phase (Appendix C).

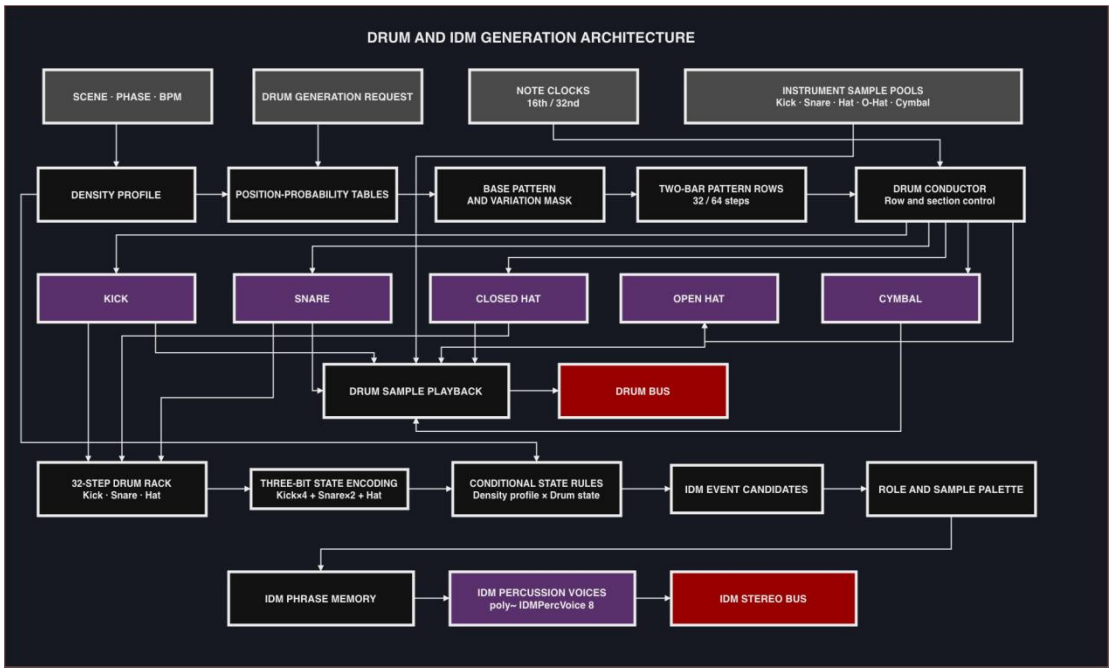
The melodic contour begins with the 'prob' chain inside 'p melodyMotifMoves'. Three successive states form a movement cell and are converted by 'coll melodyMotifMoveMap' into -2, -1, 0, +1 or +2 scale degrees. The cell is reused while the resulting degrees stay between 2 and 11, starting from the current chord root. On each bar downbeat, 'p melodyStrongBeatSnap' raises odd chord-relative offsets by one degree, placing the note on the root, third, fifth or seventh. At step 12, the cadence is limited to the chord root or third.



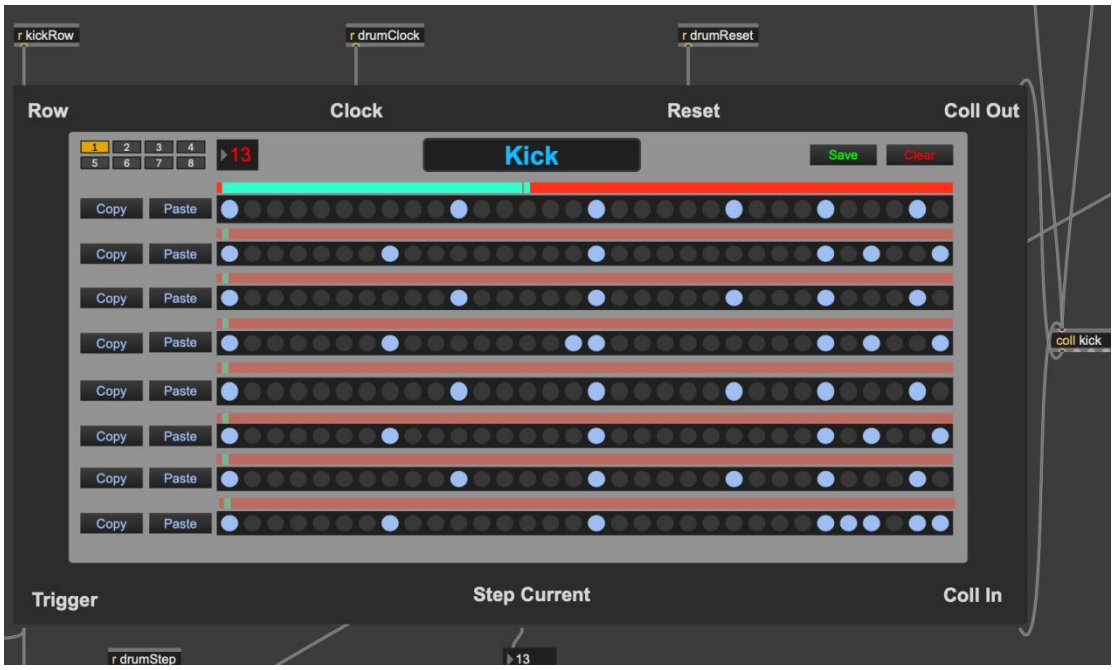
The first melody patch chose every note live. It seldom produced a phrase that I could recognise on its return. I replaced it with a 32-position capture in 'coll sectionMelodyMemory'; rests are written as part of the data. Playback follows an A-B-A-C pattern. A is unchanged. B keeps 90% of the notes and ends at step 29. C keeps 78%, ends at step 27, and has a 20% chance of raising a retained note by one scale degree.

During capture, 'p melodyLandingDetector' starts counting after four consecutive notes. Arrival on the chord root or scale degree 0 has a 55% chance of starting a rest; a ninth consecutive note forces one. 'p melodyRestScheduler' then chooses one, two, three or four whole notes with weights of 40%, 30%, 20% and 10%. The pause is stored with the phrase. Finally, 'harm_scale' and 'harm_tonicRef' convert degree to pitch before Max sends the MIDI-derived OSC value to Rack.

4.5 Drum System, Probabilistic Percussion and IDM Layer

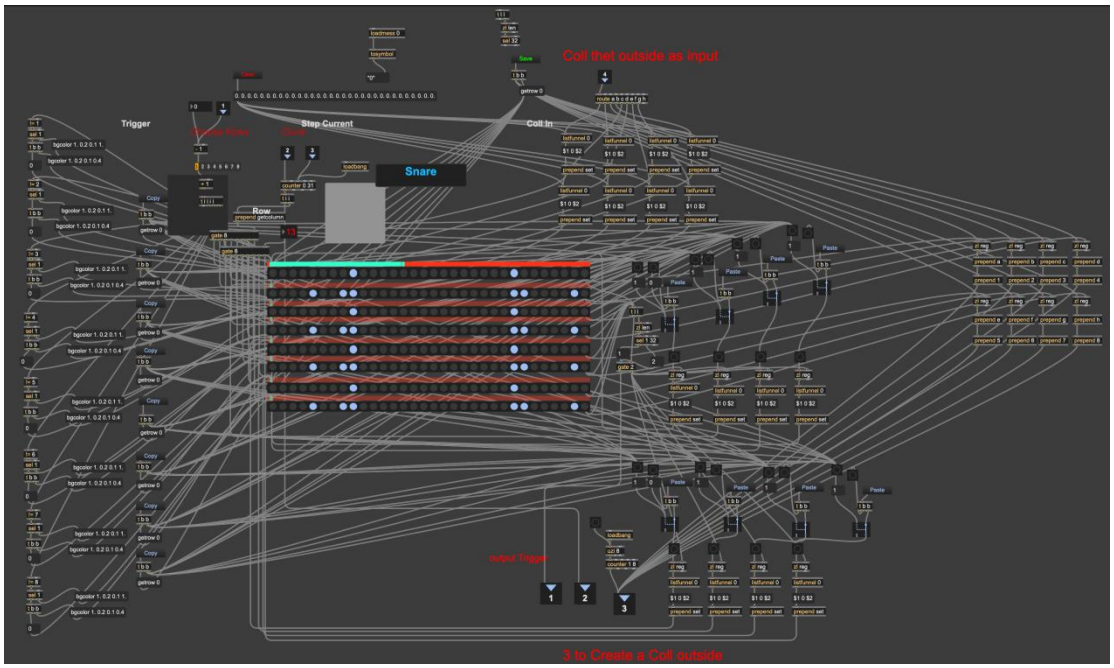


I kept the regular kit and the IDM samples as separate layers. A drum pattern is held for the section. The IDM process observes that pattern and places samples under its own scene, tempo and density conditions.



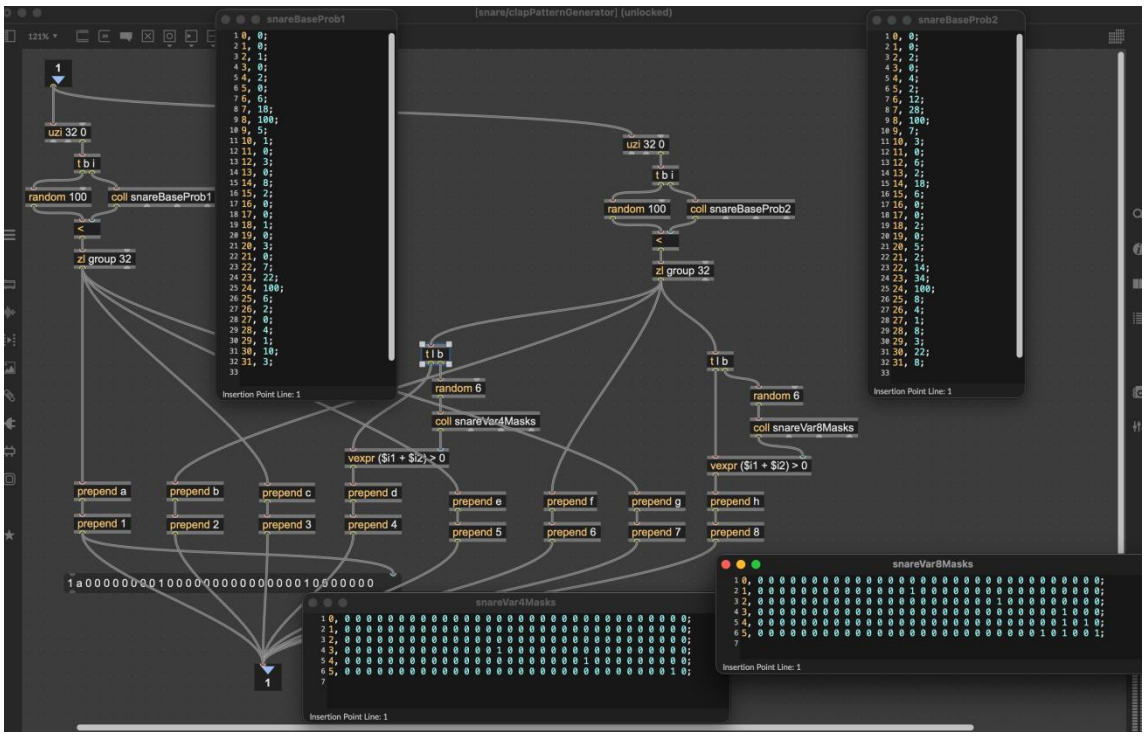
The kit uses five binary sequencers. Each has eight selectable rows of 32 steps, and I added a 64-step version for breakbeat material. Save, copy and paste remained in the interface because I used them when comparing versions of the probability tables.



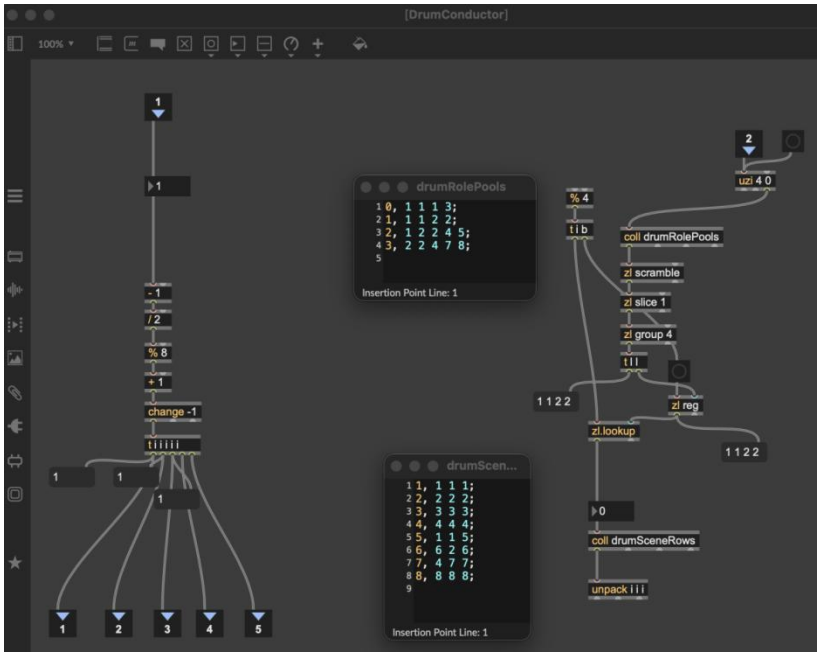


The complete sequencer subpatches are included in the submitted Max project.

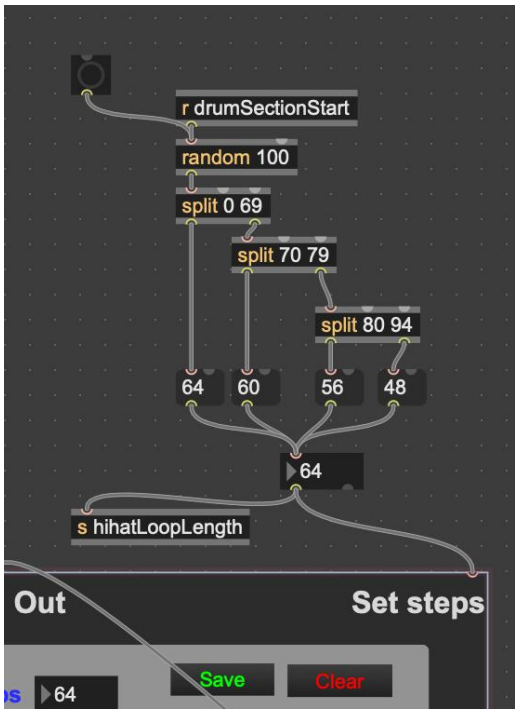
4.5.1 Pattern Generation and Section Identity



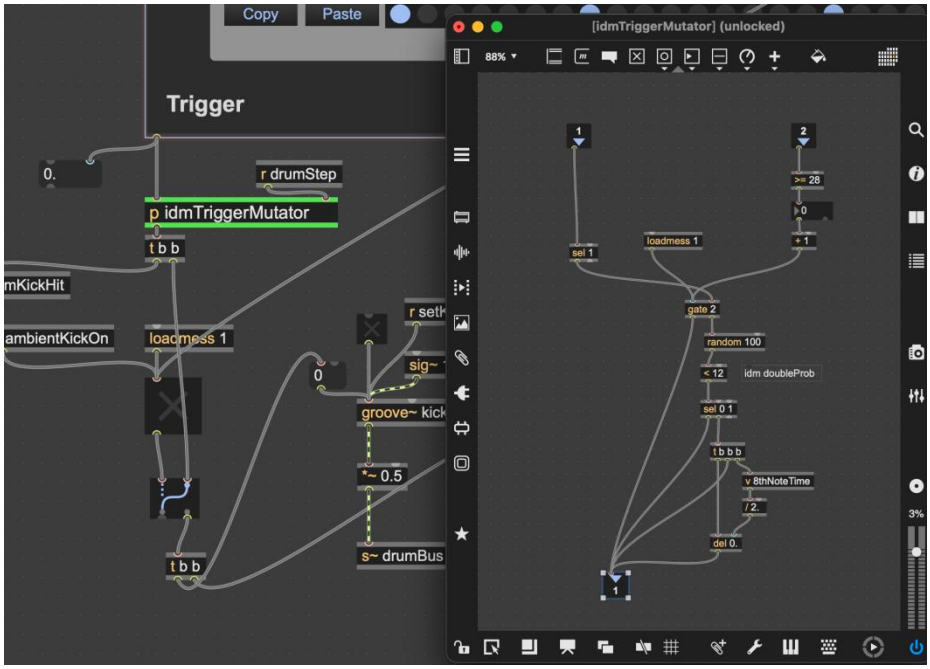
When 'section_generateRequest' arrives, every grid position runs 'random 100' and tests the result against its own threshold. A small variation mask is added with 'vexpr ($i1 + $i2) > 0'. I tried regenerating continuously, but the pulse lost its identity. The finished pattern now stays unchanged until the next section request.



'DrumConductor' reads 'barIndex' and selects one row. Since a row lasts two bars, the eight rows make a sixteen-bar cycle. The same row number is sent to all five sequencers:



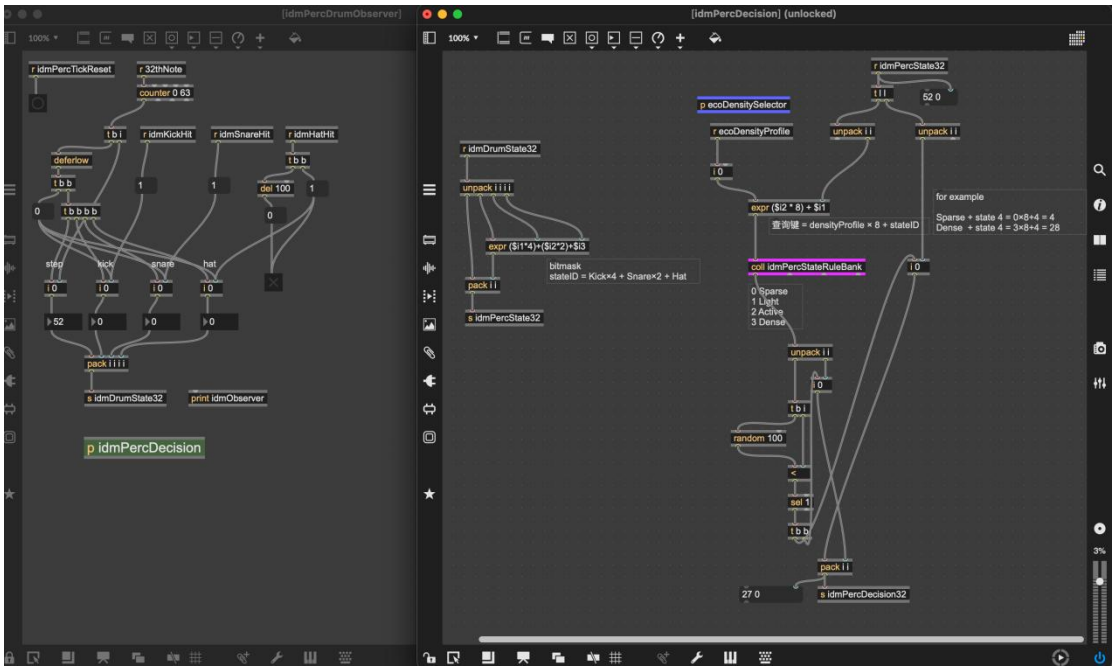
The Closed Hat is allowed to loop after 48, 56, 60 or 64 steps. Choosing one of the shorter lengths displaces it against the 64-step drum reference for part of the cycle. The other sequencers keep their original grids.



Five trigger mutators may add one doubled hit a sixteenth note after an existing attack. The stored chances are 12% for Kick, 25% for Snare/Clap,

5% for Cymbals, 5% for Open Hi-hat and 35% for Closed Hi-hat.This is the characteristic of the IDM-style drum kit.

4.5.2 Drum State Coding and Conditional IDM Decision-Making



On every thirty-second-note pulse, 'p idmPercDrumObserver' checks the current Kick, Snare/Clap and Closed Hat values. The three bits are packed into one integer:

stateID = Kick × 4 + Snare × 2 + Hi-hat

The values 0-7 identify all eight combinations of the three binary inputs.

At 'section_materialReady', the scene selects a density profile from 'coll ecoDensityPools'. BPM values of 150 or above add an upward bias. 'p idmPercDecision' then calculates ruleKey = densityProfile × 8 + stateID.

'coll idmPercStateRuleBank' returns a probability threshold and a role code for that key. A 'random 100' comparison decides whether to create an extra event. Current drum activity, scene density and tempo all affect the decision.

4.5.3 Sample Palette and Phrase Memory

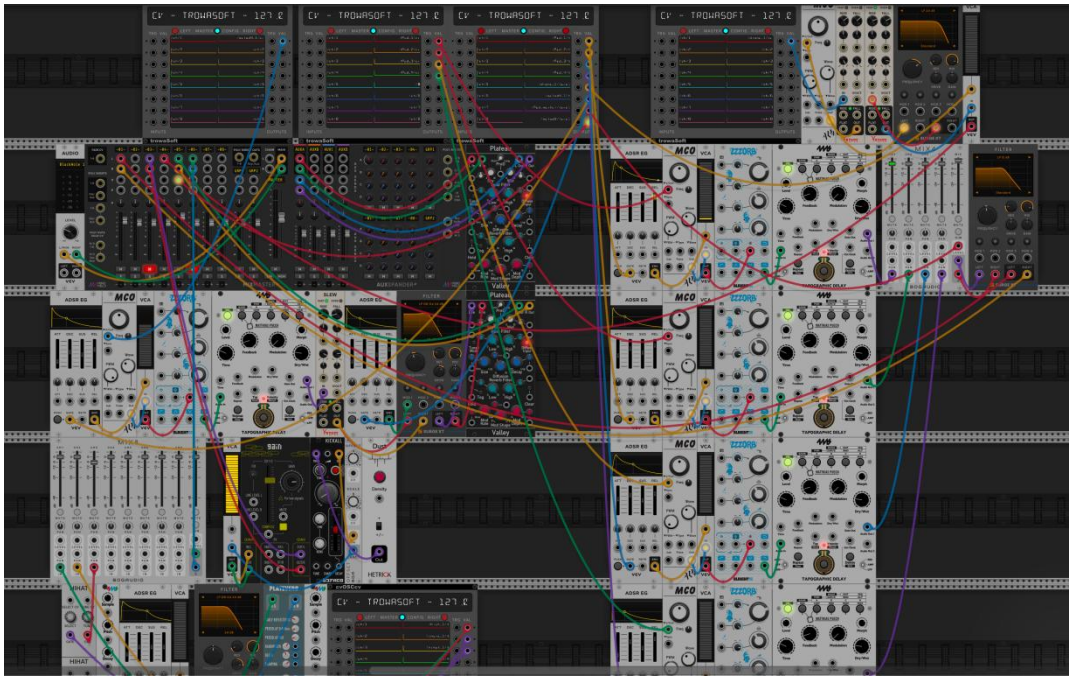
The sample pool contains 57 files in five roles: Foley/Click, Glitch, Impact/Metallic, Noise and Reverse Swell. On 'section_materialReady', 'p ecoPaletteGenerator' draws one file from each role and stores the five IDs as the section palette.

During the first 64 thirty-second-note steps, accepted IDM events are written to 'coll ecoPhrase' as step position, sample ID and playback mode. Playback then reads this phrase until new section material is generated. I used phrase storage here to stop the IDM layer sounding like an unrelated sample was chosen on every clock.

4.6 VCV Rack Synthesis and Audio Routing

4.6.1 OSC Communication between Max/MSP and VCV Rack

Max/MSP generates pitch, trigger and level data. VCV Rack converts these values into control voltages and synthesises the Pad, Melody and Drone voices. The stereo result returns to Max for mixing and recording.

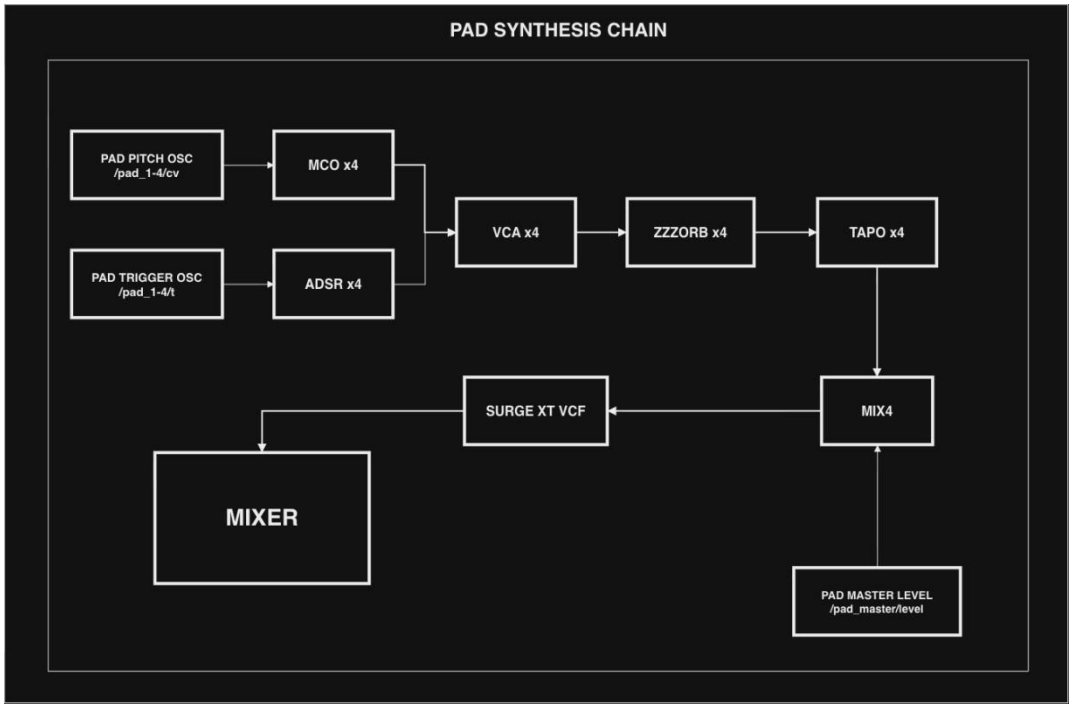


The Rack patch contains five trowaSoft 'cvOSCcv' modules. Four carry active control data from Max. The Pad voices use '/pad_1/cv' to '/pad_4/cv' for pitch and '/pad_1/t' to '/pad_4/t' for triggers. Melody uses '/melody_1/cv' and '/melody_1/t'; Drone uses '/drone_1/cv' and '/drone_1/level'. Other active paths control the master levels of the Pad and Melody layers. The fifth module retains an inactive drum-trigger branch from an earlier prototype, which the current Max patch does not transmit.

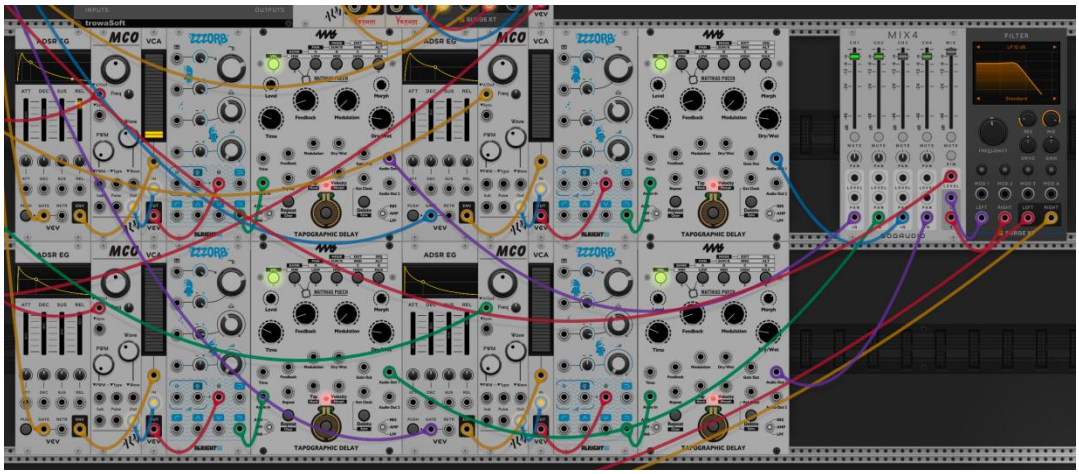
Max converts MIDI pitch values to a one-volt-per-octave scale before transmission. Binary values of 0 and 1 form trigger pulses, and continuous values control amplitude. 'cvOSCcv' converts the OSC messages into voltages for the Rack modules.

Audio follows a separate return path. 'MixMasterJr' feeds a stereo signal to 'AudioInterface2', which is configured for BlackHole 16ch. Max receives channels 1 and 2 through 'adc~ 1 2' and adds them to its sample-based layers.

4.6.2 Polyphonic Pad Synthesis



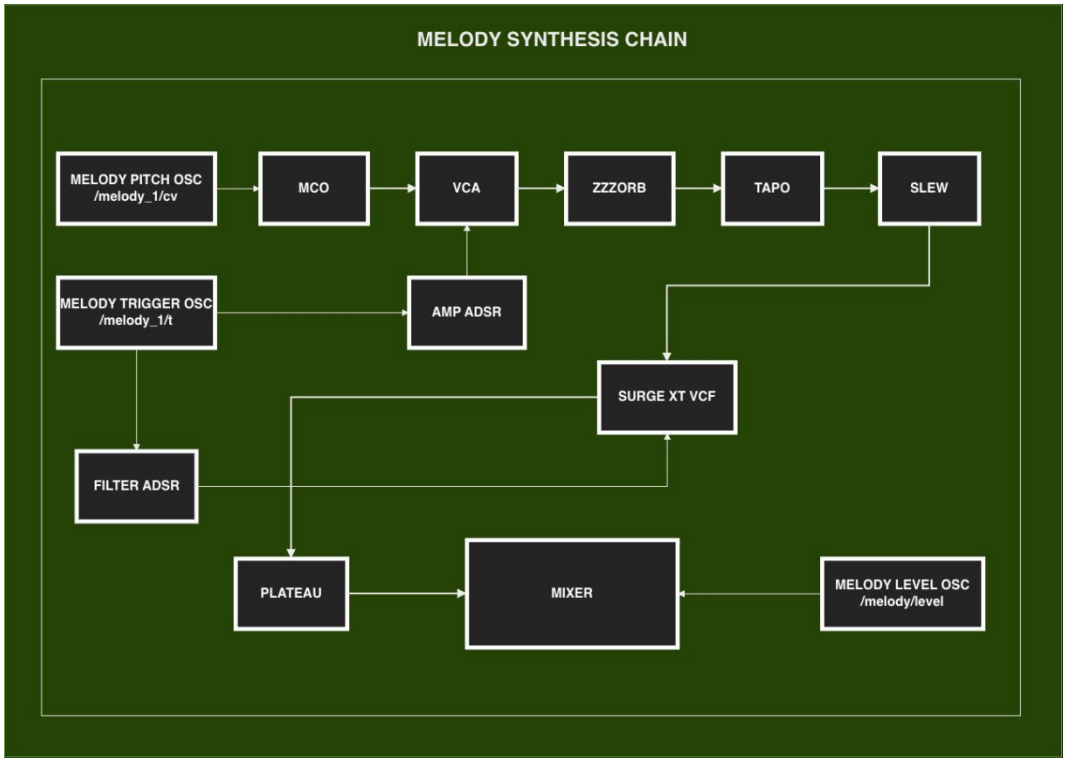
Pad synthesis is four-voice polyphony. Each voice begins with an ALM 'MCO', 'ADSR' and 'VCA', driven by its own pitch and trigger pair from Max.

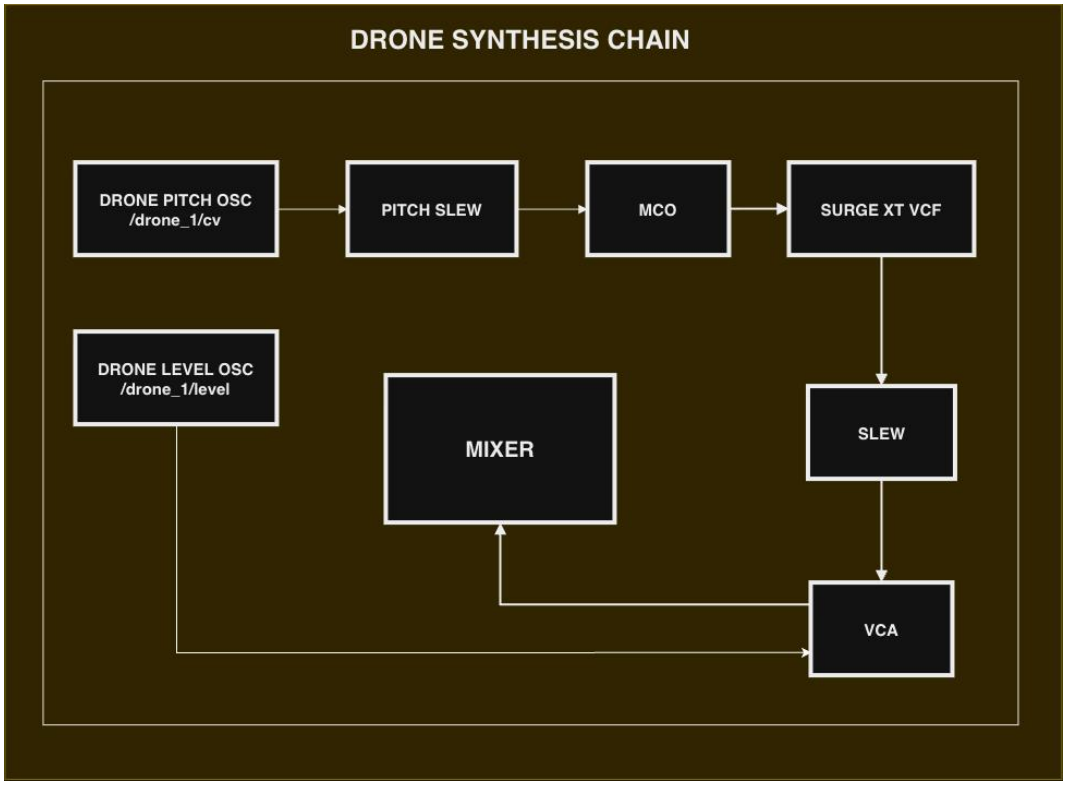


After each voice, a 'Zzzorb' filter feeds a 4 ms 'Tapo' delay. 'Mix4' sums the four paths, followed by one Surge XT 'VCF' and 'MixMasterJr'. The bus level comes from Max through '/pad_master/level'.

Rack receives pitches already resolved from MIDI. Mode selection, inversion and voice leading have all been completed in Max.

4.6.3 Melody and Drone Synthesis





The Melody voice uses one ALM 'MCO'. Pitch arrives at '/melody_1/cv', and '/melody_1/t' fires the amplitude and filter envelopes. Its audio path is 'VCA', 'Zzzorb', 'Tapo', 'Slew', Surge XT 'VCF', 'Plateau' and finally 'MixMasterJr'. Max sets the output through '/melody/level'.

The melody signal passes through:



MCO → VCA → Zzzorb → Tapo → Slew → Surge XT 'VCF' → Plateau

The Drone runs continuously and contains two 'Slew' stages. One smooths the OSC pitch before it reaches the 'MCO'. The other smooths the filtered audio on the way to the final 'VCA'.



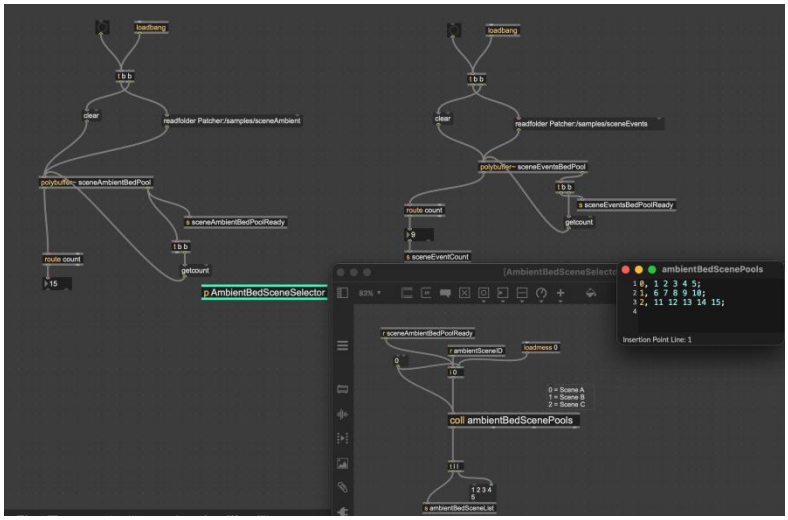
OSC pitch → Slew → MCO → Surge XT 'VCF' → Slew → VCA

'/drone_1/level' controls the 'VCA' directly, so the Drone can enter and leave gradually without a note trigger.

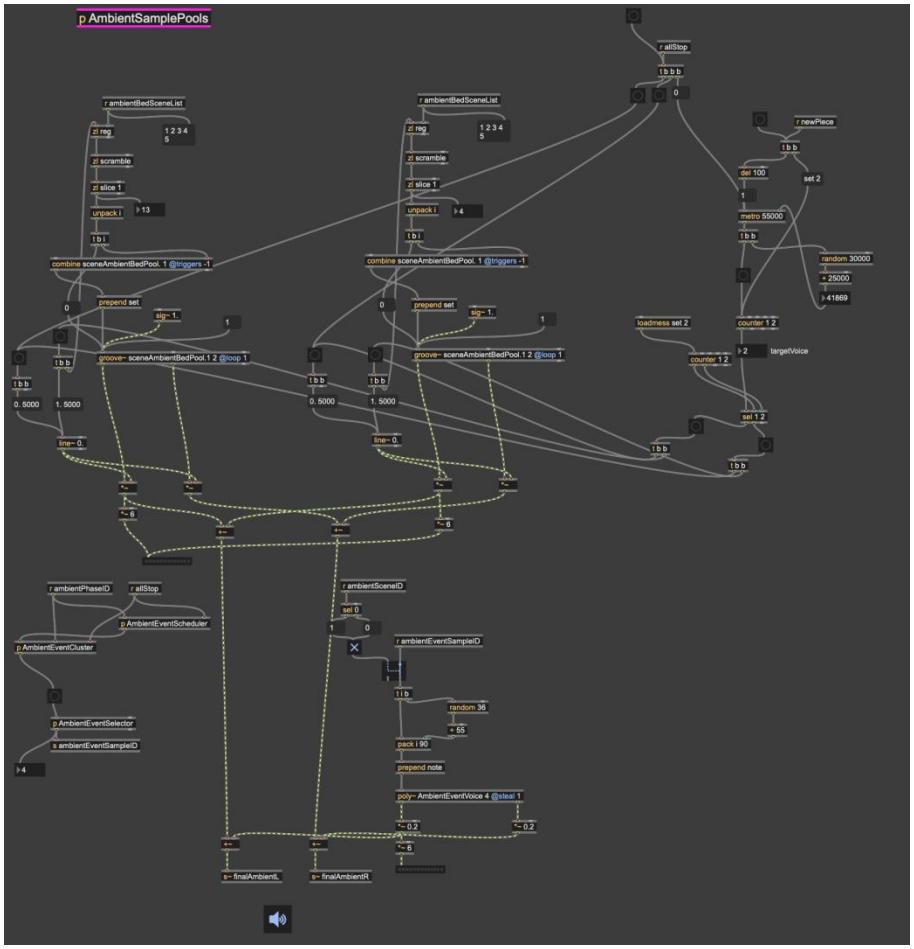
4.7 Sample-Based Soundscape

Two sample systems surround the generated parts: long ambient beds and short environmental events. Both load material through 'polybuffer~' and feed the stereo 'finalAmbientL' and 'finalAmbientR' paths.

4.7.1 Scene-Based Ambient Beds



At start-up, 'AmbientSamplePools' loads fifteen long recordings from 'sceneAmbient'. The embedded 'ambientBedScenePools' table assigns five files to each scene: Forest (1-5), Rain (6-10) and Steampunk (11-15).



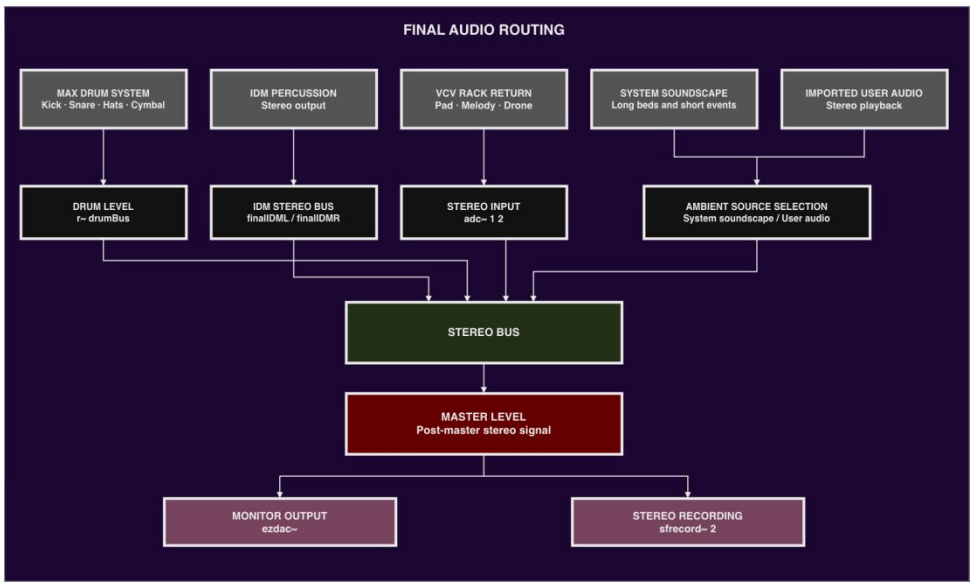
Two stereo 'groove~' players alternate between files from the selected scene. A new file is chosen every 25-55 seconds and crossfaded over five seconds. This timer is independent of the metrical clock. 'allStop' ends the timer and fades out both players.

4.7.2 Phase-Dependent Environmental Events

A separate pool contains nine short Forest recordings, including bird and monkey sounds. 'ambientPhaseID' enables them only during FLOW and BLOOM. FLOW uses intervals of 12-28 seconds; BLOOM uses 5-12 seconds and has a 35% chance of adding a second event 0.4-1.8 seconds later.

'AmbientEventSelector' uses 'urn' to avoid repetition until the pool is exhausted. Events receive a velocity of 55-90 and are sent to 'poly~ AmbientEventVoice 4', allowing four overlaps. This layer is currently limited to Forest because the short-event pool contains no Rain or Steampunk material.

4.8 User Media, Mixing, Recording and Interface



I added a media-import function so that users can place their own stereo audio recording alongside the generated music. The imported file follows the main transport, and I use its duration to set the length of a Timed run. While it is playing, I mute the internal environmental recording. I then mix the imported audio with the drums, IDM material, ambient layer and VCV Rack return. Each source has its own level control, and the complete stereo output is recorded through 'sfrecord~ 2'. For performance, I arranged the essential controls in Presentation view and kept the BPM, form position, active parts, harmony and latest Melody note visible.

5 Evaluation and Critical Reflection

5.1 Musical Results

I did not use a numerical score. I listened to the patch run several times and paid attention to how long I could still follow what it was doing. I listened for material coming back and for the points where one section gave way to another. I repeated this with each of the three scene settings.

The runs I wanted to keep usually had less material in them. I used a small chord set, saved one melodic phrase, and let a two-bar drum pattern return.

Earlier versions regenerated the harmony and melody more often. They sounded busier, but I soon lost track of what belonged to the section. I had better results when the notes stayed in place and most of the change came from orchestration.

The three scenes differ most clearly through their samples and tempo settings. Forest has more foreground detail because I enabled its short-event layer. Rain depends more on its bed recordings and tempo pool, while its voice masks remain close to those of Forest. For Steampunk, I used mechanical samples and allowed faster tempi. The IDM layer is more noticeable in this setting.

5.2 Effectiveness of Structured Probability

Of all the revisions, section memory changed the listening result most clearly.At a section boundary, a small chord collection is saved and replayed through one pattern. Melody and drums keep shorter forms of the same memory. After I made this change, later probabilistic events were heard in relation to an existing passage. Before it, they often sounded unrelated.

Slower change comes from the formal controller. Locally, harmonic functions, the approved modulation routes and minimum-movement Pad voicings restrict pitch. These limits proved more dependable in testing than choosing from the entire chord space on every event. A uniform chance at every drum position usually weakened the pulse. I kept several attacks stable with position weights and let the variation masks change only part of a pattern. Before choosing a sample, the IDM layer also reads the current Kick, Snare and Closed Hat state. Some of its activity is consequently tied to the kit.

When material returns, it comes from what the system has stored. Some details change with each repetition. Over the course of a section, the current formal state controls how busy the music becomes.

5.3 Sound-Design Evaluation

In my final version, the environmental bed usually establishes the scene first. Short events sit closer to the foreground, and I found they were most effective when used sparingly.

Separate Kick, Snare and Hat pools preserved the identity of each drum role when the samples changed. Foley, metallic hits, noise and reverse gestures

widened the palette. Some combinations still overlapped the Pad or bed and needed a manual level change.

The sustained voices gained useful internal movement from VCV Rack, especially its filters, envelopes, delay and slow modulation. But the cost is practical: another application and an audio route have to be installed before the piece can run.

5.4 Limitations

The authored 'coll' tables define a finite output space. This helped stability, but recurring harmonic, rhythmic and formal patterns become noticeable over many runs.The scene libraries are uneven as well. Forest has both long beds and foreground events. Rain and Steampunk have fewer detailed event recordings.

Imported audio sets the duration and joins the mix. The patch does not analyse its spectrum, rhythm or semantic content. The rules still permit combinations with crowded registers or an uneven mix. I used manual listening to identify these cases. A future version could coordinate density, register and spectral occupancy before committing an event. Distribution is also complicated by the VCV Rack dependency and its audio routing.

5.5 Future Development

A self-contained version would replace the three VCV voices with Max/MSP instruments. RNBO could then cover compatible audio processing, while a TypeScript implementation held the state tables and memories in a browser. This has not been tested in the current project.

I also considered natural-language control. A small JSON interface could expose scene, density and duration values to a Python service. A language model could propose those values, but the existing patch would validate them and perform the score.At present, all transition and density weights are hand-authored. A later study could estimate some of them from symbolic or audio corpora and compare the result with the current tables.

6 Reflection and Conclusion

6.1 Answer to the Research Question

The research question was:
How can structured probability be used to generate ambient music that remains varied while retaining coherent musical form?

Testing changed my answer to the research question. The system became easier to control once I divided its decisions by timescale. Weighted draws handle immediate choices such as notes, drum positions and samples. Stored chords, phrases and beats give each section material to return to. Three first-order chains guide harmonic function, melodic movement and Pad shape. Scene profiles and the five formal states organise the larger duration and instrumentation.

After listening to repeated runs, I stopped treating more variation as an improvement. The music was easier to follow when generated material returned several times. I therefore kept it for the length of a section and replaced it only at a section boundary or a change of formal state.

6.2 Creative and Technical Reflection

The early patches moved on before I had time to recognise what they had generated. I changed this by keeping the chords, melody and drum pattern for the whole section. A later generation request replaces them, so the next run can use different material.

I came to treat the probability tables as part of the score. Listening led to several changes in weights, routes and rhythm templates, especially when harmony wandered or the percussion became crowded. The values in the final patch retain that history.

The finished Max/MSP patch coordinates harmony, melody, drums and samples; VCV Rack supplies the pitched synthesis. Clock and state messages keep those modules together. Evolve runs until stopped. Timed follows either a duration entered by the user or the length of imported audio.

References

Carnovalini, F. and Rodà, A. (2020) ‘Computational Creativity and Music Generation Systems: An Introduction to the State of the Art’, Frontiers in Artificial Intelligence, 3, Article 14. doi: 10.3389/frai.2020.00014. https://www.frontiersin.org/journals/artificial-intelligence/articles/10.3389/frai.2020.00014/full(Accessed: 12 August 2026).

Collins, N. (2008) ‘The Analysis of Generative Music Programs’, Organised Sound, 13(3), pp. 237–248. doi: 10.1017/S1355771808000332. https://www.frontiersin.org/journals/artificial-intelligence/articles/10.3389/frai.2020.00014/full(Accessed: 12 August 2026).

Fell, M. and Gilmore, J. (2012) Composing with Process: Perspectives on Generative and Systems Music, episodes 7.1 and 8.1. Ràdio Web MACBA. Available at: https://rwm.macba.cat/en/taxonomy_serie_podcast/composing-with-process/ (Accessed: 12 August 2026).

Xenakis, I. (1992) Formalized Music: Thought and Mathematics in Composition. Revised edn. Additional material compiled and edited by S. Kanach. Stuyvesant, NY: Pendragon Press.

Appendix A: AI Use Statement

AI-assisted tools were used to clarify selected academic concepts, including Markov chains and algorithmic composition, discuss possible solutions to software bugs and organise rule-table data. The author designed the system, wrote its rules, implemented and tested it, and made the final editorial decisions.
AI-assisted tools were also used to refine academic vocabulary, improve sentence fluency and check selected factual claims.

Models: ChatGPT 5.6 Sol; Claude Opus 4.8

Appendix B: Third-Party Materials and Credits

Audio-material sources, licences and processing notes for the ambient, IDM and drum libraries are supplied in the accompanying credits document. The same document lists the VCV Rack modules, diagrams.net (draw.io) and the BlackHole virtual audio driver used by the project.

The Cycling ’74 Max Reference was consulted throughout development for the syntax and behaviour of the Max objects used in the patch. Available at:
https://docs.cycling74.com/reference/ (Accessed:15 August 2026)

The VCV Rack Library was consulted during development Available at:
https://library.vcvrack.com/ (Accessed: 16 August 2026)

Appendix C: Probability and Mapping Tables

Appendix C contains the full technical tables cited in Chapter 4. The main text includes only the values needed to explain each mechanism.

Table C.1. Pad-Chord Transition Matrix

Current state	0	1	2	3	4	5
0 full7	33%	22%	17%	6%	11%	11%
1 shell7	33%	33%	13%	7%	7%	7%
2 no3_7	29%	12%	29%	6%	12%	12%
3 triad	47%	20%	13%	6%	7%	7%
4 sus2_7	50%	19%	13%	6%	6%	6%
5 sus4_7	50%	19%	13%	6%	6%	6%

Table C.2. Pad-Chord State Mapping

State ID	Chord shape	Alteration type	Active mask
0	Full seventh chord	0	1 1 1 1
1	Seventh chord without fifth	0	1 1 0 1
2	Seventh chord without third	0	1 0 1 1
3	Triad	0	1 1 1 0
4	Sus2 seventh chord	1	1 1 1 1
5	Sus4 seventh chord	2	1 1 1 1

Table C.3. 'phaseMinSections'

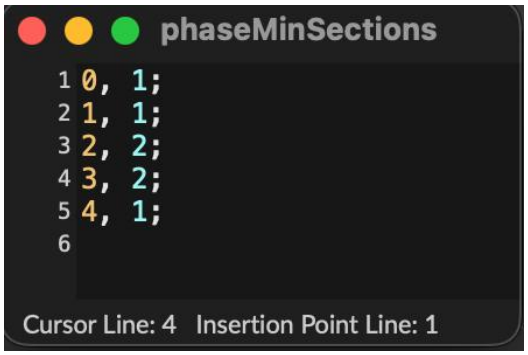


Table C.4. Scene Entry Plan Allocation

Scene Entry Plan Allocation		
Scene	Scene ID	Available entry plans
Forest	0	0, 1, 2
Rain	1	3, 4, 5
Steampunk	2	6, 7, 8

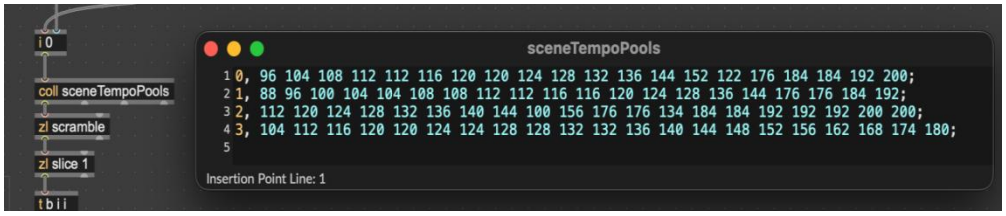


Table C.5. Voice-mask lookup and distribution to eight instrumental layers

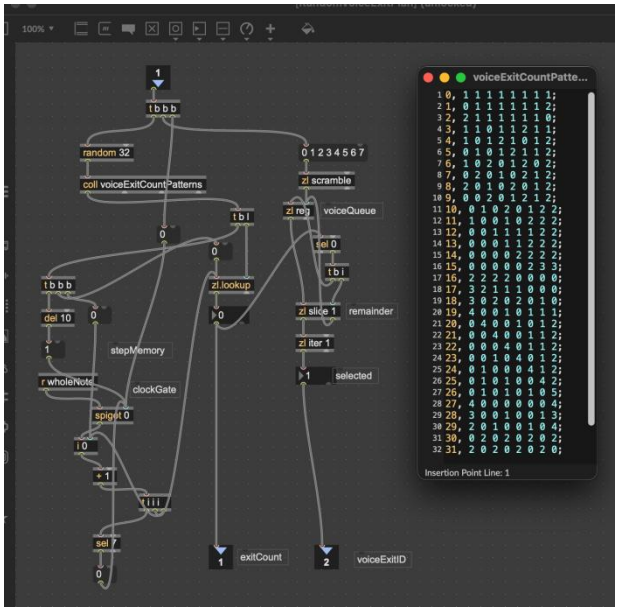


Table C.6. Weighted transition between four harmonic-function states

State ID	Current Function	Tonic	Predominant	Dominant	Color	Row Total
0	Tonic	40%	40%	10%	10%	100%
1	Predominant	15%	15%	60%	10%	100%
2	Dominant	75%	5%	10%	10%	100%
3	Color	35%	30%	25%	10%	100%

Table C.7. Construction of a seventh chord from the selected scale-degree root

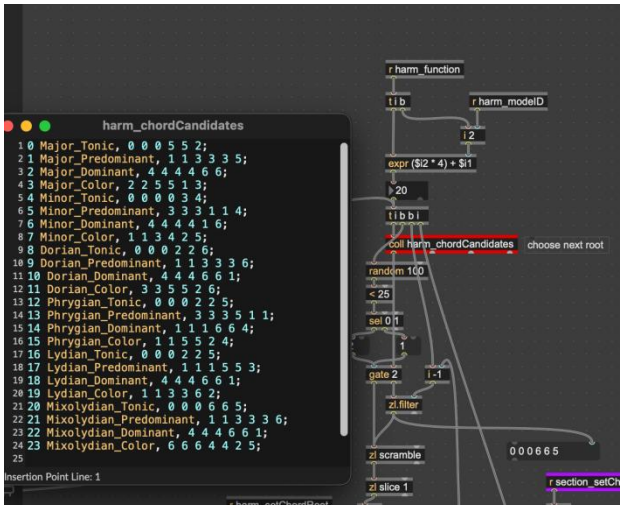


Table C.8. Lookup and application of an authorised modulation route

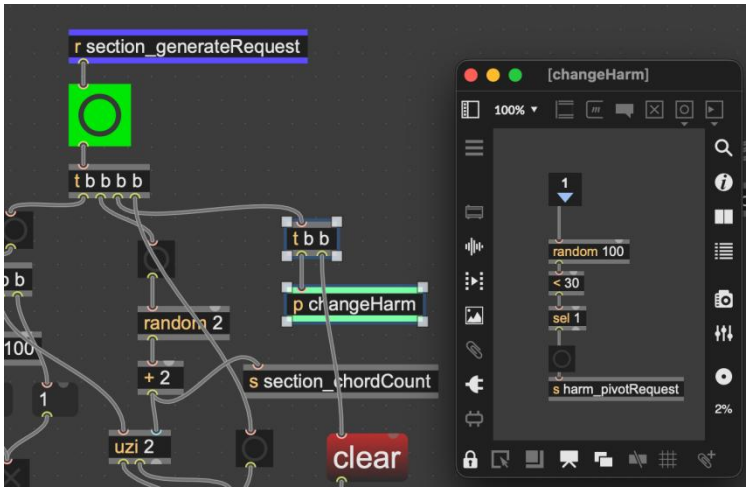
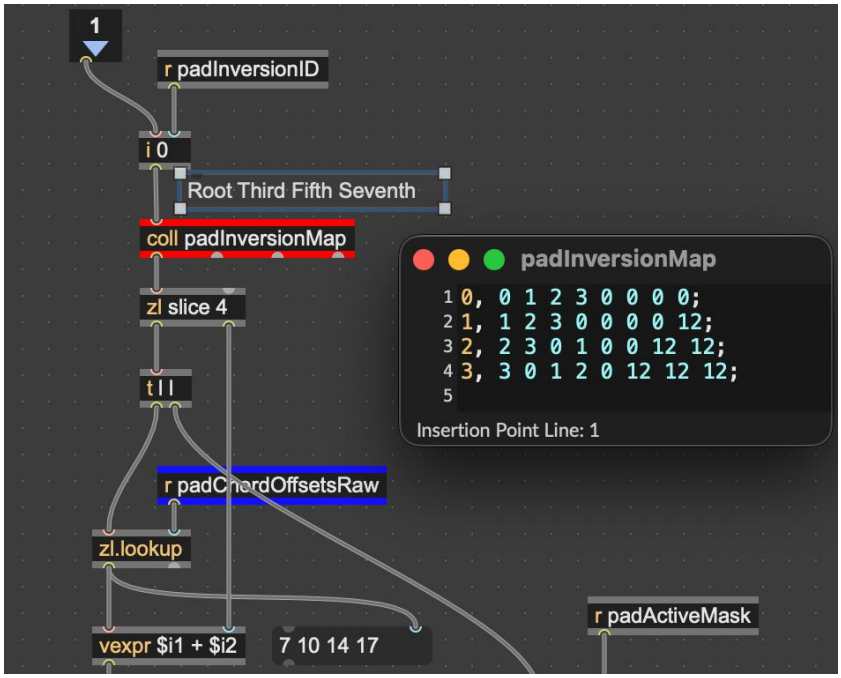


Table C.9. 'coll padInversionMap' and 'p padInversionBuilder'



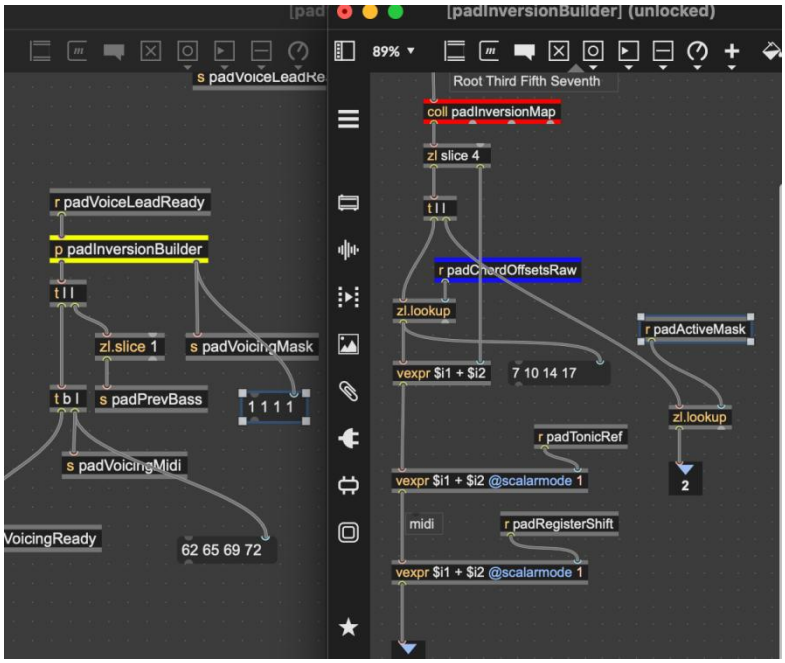


Table C.10. Melody-density weighting by formal phase

Formal phase	Sparse patterns	Medium patterns	Dense patterns
STILL	75.0%	25.0%	0.0%
EMERGE	37.5%	50.0%	12.5%
FLOW	12.5%	50.0%	37.5%
BLOOM	0.0%	25.0%	75.0%
DISSOLVE	50.0%	37.5%	12.5%